A Project Report on

CENTRICA CUSTOMER CHURN & LOYALTY ANALYTICS PLATFORM WITH RULE-BASED SCORING &SCD

PREPARED BY:

ASSOCIATE ID	ASSOCIATE NAME
2480302	Harshan Ramanathan
2479635	Vishal P
2480255	Aleemullah Hussain
2480148	Anu Anil
2479525	Arun Kumaran A
2480147	Arun S K
2480271	Karthika Nagaraj N A
2480173	Subhadip Singha
2479610	Thamizhini K R
2479698	Vethika Venkatraman

GUIDED BY:

Mentor

Kavin Kumar G
Associate
AIA- Cloud Data Integration

Coach

Viveka Preethi
GenC Certified Coach, Data
Human Resource – GenC

CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Introduction 06

1.2 Objective 06

1.3 Data source 07

1.4 Project Scope and Outline 08

CHAPTER 2: ARCHITECTURE

2.1 Architecture Diagram 09

2.2 Overview of Azure storage solutions 11

2.3 Overview of Azure Data Factory 12

CHAPTER 3: ENVIRONMENTAL SET-UP

3.1 Creating Azure Data Factory 13

3.2 Creating Azure Storage Account 15

3.3 Data Lake Gen 2 account via Azure portal 16

3.4 Azure SQL Database 18

3.5 Create Linked Service 19

CHAPTER 4: IMPLEMENTATION

4.1 Bronze Layer Transformation 24

4.2 Silver Layer Transformation 25

4.3 SCD Type 2 Implementation 29

4.4 Gold Layer Transformation 32

CHAPTER 5: RESULTS 38

CHAPTER 6: CONCLUSION 41

LIST OF ABBREVATIONS

TERM	DESCRIPTION
ADF	Azure Data Factory
ADLS	Azure Data Lake Storage
ADLS Gen2	Azure Data Lake Storage Generation 2
SQL	Structured Query Language
DB	Database
ETL	Extract, Transform, Load
ELT	Extract, Load, Transform
SCD	Slowly Changing Dimension
SCD Type 2	Slowly Changing Dimension – Type 2
CRM	Customer Relationship Management
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
API	Application Programming Interface
HTTP	Hyper Text Transfer Protocol
CSV	Comma Separated Values
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
IR	Integration Runtime
VM	Virtual Machine

LIST OF FIGURES

Fig 2.1	Architecture Diagram
Fig 3.1	Data Factory Overview
Fig.3.2	Azure Data Factory studio
Fig.3.3	Azure Storage Account overview
Fig.3.4	Data Lake Gen 2 Storage
Fig.3.5	Azure SQL Database
Fig.3.6	Linked Service in ADF
Fig.3.7	Data in Folder
Fig.4.1	Bronze layer pipeline in ADF
Fig.4.2	Silver Layer Data Transformation in ADF
Fig.4.3	Customer Interactions Transformation in ADF
Fig.4.4	Customer Master Transformation in ADF
Fig.4.5	Energy Usage Transformation in ADF
Fig.4.6	SCD Type 2 Implementation in df_dimcustomer
Fig4.7	SCD Type 2 Implementation in df_dimcontract
Fig 4.8	Data Flow Implementation in df_dimcustomer
Fig 4.9	Data Flow Implementation in df_dimcontract
Fig 4.10	Pipeline pl_bronze_to_gold
Fig 4.11	Gold Implemention Pipeline
Fig 4.12	Gold Implementation Dataflow
Fig 5.1	Dimension table dim_contract
Fig 5.2	Dimension table dim_customer
Fig 5.3	Fact table ds_factbillingbehaviour
Fig 5.4	Fact table ds_customerrisk
Fig 5.5	Fact table ds_customerinteractions
Fig 5.6	Fact table ds_usagebehaviour

1: INTRODUCTION

1.1 Introduction:

The energy and utilities sector is undergoing rapid transformation due to increasing competition, fluctuating energy prices, and evolving customer expectations. In such a dynamic environment, retaining existing customers has become as crucial as acquiring new ones. Customer churn directly impacts revenue, brand reputation, and long-term business sustainability for energy service providers.

Centrica, operating within this highly competitive energy market, faces challenges related to customer retention caused by factors such as pricing volatility, contract expirations, billing behaviour, service quality issues, and external market disruptions. Customerrelated data is distributed across multiple systems, making it difficult to obtain a consolidated and timely view of customer behaviour- and risk.

The Energy Solution project aims to address these challenges by building a centralized analytics platform using modern cloud technologies. The solution focuses on integrating customer, billing, usage, interaction, and external market data to analyze customer behaviour and proactively identify churn risk and loyalty levels. By leveraging rulebased scoring mechanisms and historical tracking of customer attributes, the project enables -datadriven -decisionmaking- and supports proactive customer retention strategies.

1.2 Objectives

The primary objective of the Energy Solution project is to design and implement a scalable and automated analytics platform that enables Centrica to proactively manage customer churn and improve customer loyalty.

The specific objectives of the project are:

- To consolidate data from multiple source systems into a centralized data platform.
- To cleanse, standardize, and transform raw data into analytics-ready datasets.
- To maintain historical changes in customer, contract, and market attributes using Slowly Changing Dimension (SCD) Type 2 methodology.
- To calculate churn risk percentage and loyalty percentage using configurable rule-based scoring logic.
- To deliver reliable and consistent Gold-layer fact and dimension tables for reporting and

analysis.

- To enable automated daily orchestration for timely insights and decision support.

By achieving these objectives, the project helps the organization shift from a reactive approach to a proactive, predictive, and risk-based customer retention model.

1.3 Data Sources:

The Energy Solution project utilizes multiple internal and external data sources to gain a complete and unified view of customer behaviour, operational trends, and market influences. These diverse data sources enable accurate churn risk prediction and loyalty assessment by capturing both customerspecific and external environmental factors.

The major data sources used in this project are described below:

1.3.1 Customer Master Data

Customer master data is sourced from Customer Relationship Management (CRM) systems. This data includes customer demographics, personal details, tenure, region, and contract information. It serves as the foundation for customerlevel analytics and is essential for identifying longterm behavioural patterns and retention trends.

1.3.2 Billing and Payment Data

Billing data is obtained from internal billing and finance systems. It contains information related to monthly invoices, billing amounts, arrears, payment delays, and outstanding balances. This data plays a critical role in identifying risk markers such as late payments and billing fluctuations, which directly influence churn risk.

1.3.3 Energy Usage Data

Energy usage data is collected from metering systems, including smart meters and traditional meters. It provides consumption summaries over daily, weekly, or monthly periods. Usage patterns are analyzed to detect abnormal consumption behaviour, which may indicate customer dissatisfaction or potential service issues.

1.3.4 Customer Interaction Data

Customer interaction data is sourced from service management and customer support platforms. This data includes customer complaints, service requests, call center interactions, and digital

engagement records. Frequent complaints or repeated service issues are considered key indicators of reduced customer satisfaction and increased churn risk.

1.3.5 External Market Indicator Data

External market data is sourced from third-party providers and market intelligence platforms. It includes energy price volatility indices and geopolitical risk indicators. These external factors are incorporated into the analytics model to assess their impact on customer sentiment and churn behaviour during periods of market instability.

1.3.6 Data Storage Layer

All source data is ingested into Azure Data Lake Gen2 and initially stored in the Bronze Layer in raw format. After validation, cleansing, and transformation, the data is promoted to the Silver Layer, and finally to the Gold Layer for analytics and reporting purposes.

1.4 Project Scope & Outline:

The Project is focused on EU countries and the UK only, using data from EU sources to maintain consistency throughout the course. Primarily focus on building the data platform using Azure Data Factory. Data collection, data ingestion into the data lake, data transformation concepts are covered, and populating the data warehouse for reporting purposes is being done

The scope covers the implementation of a layered data architecture consisting of Bronze, Silver, and Gold layers using Azure Data Lake Storage Gen2. It also includes the use of Azure Data Factory for data ingestion, transformation, orchestration, and historical tracking using Slowly Changing Dimension (SCD) Type 2 methodology. Rule-based churn and loyalty scoring logic is applied to generate meaningful insights for business decision-making. The project does not include real-time processing, machine learning model implementation, or automated customer outreach mechanisms.

.

2: ARCHITECTURE:

2.1 SOLUTION ARCHITECTURE DIAGRAM:

The solution architecture illustrates the end-to-end data flow from multiple source systems through Azure Data Factory into Azure Data Lake using a Bronze, Silver, and Gold layered approach. It highlights data ingestion, transformation using mapping data flows, SCD Type 2 processing, and final analytics consumption through Power BI.

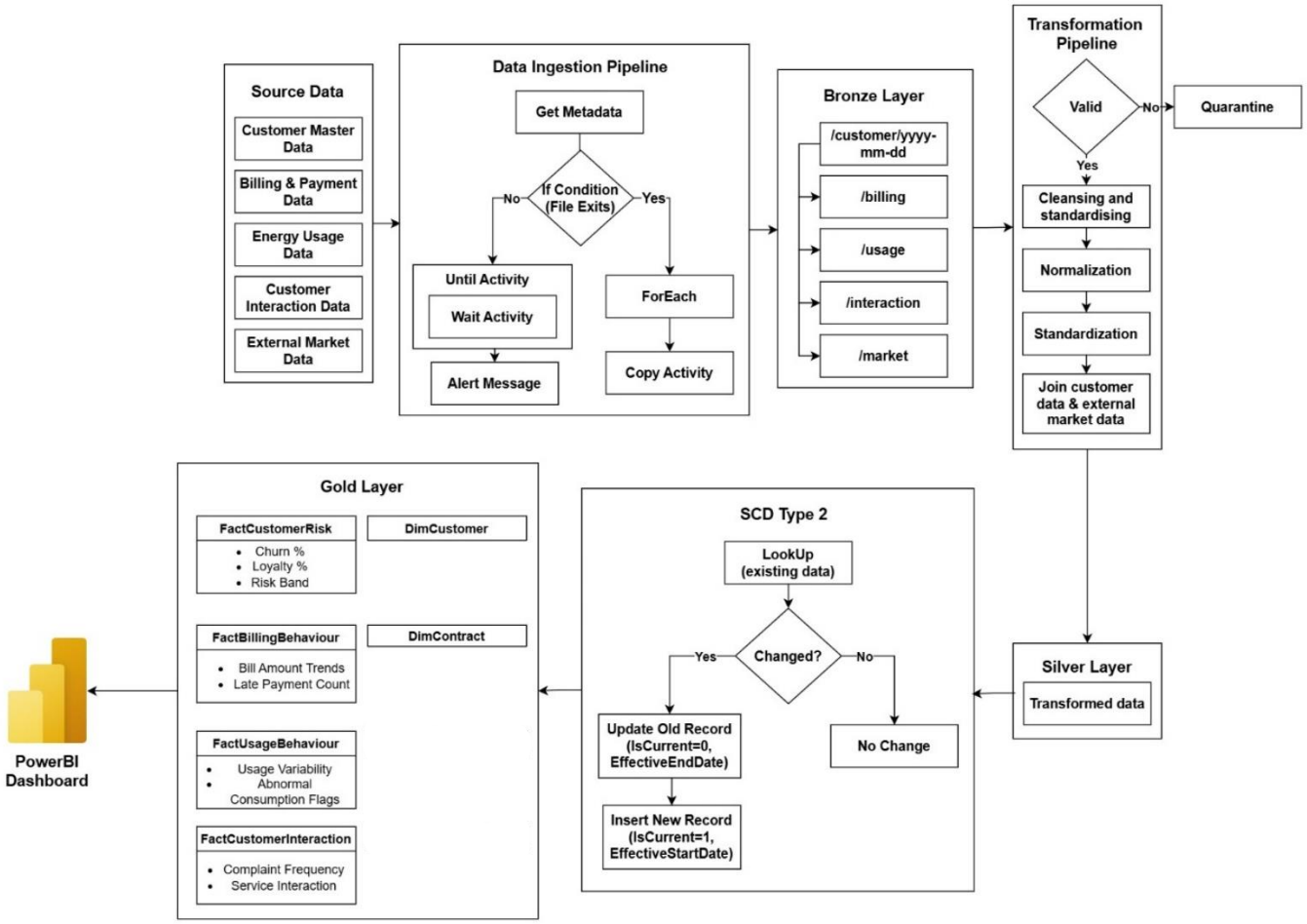


Fig.2.1: Architecture Diagram

Data Ingestion:

The data ingestion layer is responsible for extracting data from multiple internal and external sources relevant to customer behaviour and market conditions.

1. Customer Master Data

Customer master data is sourced from internal CRM systems, containing customer demographic details, tenure information, region, and contract attributes. This data is ingested using ADF Copy Activity and validated using Get Metadata activities to ensure file availability and schema compliance.

2. Billing and Payment Data

Billing and payment data is sourced from internal billing systems. This includes invoice details, billing amounts, outstanding balances, and payment delay information. The ingestion process ensures periodic batch data arrival and validates mandatory fields before loading the data into the data lake.

3. Energy Usage Data

Energy usage data is ingested from metering systems, which capture electricity and gas consumption summaries. Data ingestion ensures alignment with billing cycles and supports further analytics related to abnormal consumption behaviour.

4. Customer Interaction Data

Customer interaction data is sourced from customer service and support platforms, including complaints, service calls, and digital interactions. This data helps measure customer sentiment and dissatisfaction indicators.

5. External Market Data

External market data includes energy price volatility indices and market disruption indicators. These datasets are ingested to factor in environmental and geopolitical influences that may impact customer churn behaviour.

Data Lake Storage

All ingested datasets are initially stored in Azure Data Lake Storage Gen2 (ADLS Gen2) under the Bronze Layer, preserving raw data in its original format for traceability and audit purposes.

Data Transformation Using Data Flows

ADF Mapping Data Flows

Azure Data Factory Mapping Data Flows are used to perform simple to medium-complexity transformations without writing code. These transformations include:

- Data validation and cleansing
- Standardization of customer and billing attributes
- Normalization of usage data

- Categorization of interaction data
- Enrichment using external market indicators

Mapping Data Flows are chosen for their scalability, visual interface, and ease of maintenance, making them suitable for enterprise analytics workflows.

Any records failing validation rules are routed to a Quarantine/Error folder, ensuring data quality and compliance.

Data Storage

Azure Data Lake Storage Gen2

Azure Data Lake Gen2 serves as the primary data lake for the project and stores data across three layers:

- **Bronze Layer** – Raw ingested data
- **Silver Layer** – Transformed and cleansed data
- **Gold Layer** – Analytics-ready fact and dimension tables

It supports high-volume storage, hierarchical namespaces, and seamless integration with Azure analytics services.

Azure SQL Database

For the final data warehouse layer, Azure SQL Database is used to store Gold-layer fact and dimension tables. While Azure Synapse Analytics is ideal for very large-scale datasets, Azure SQL Database is sufficient and cost-effective for the moderate-sized datasets processed in this project.These structured tables are optimized for reporting and serve as the data source for Power BI dashboards.

2.2 Overview of Azure Storage Solutions

Azure provides a wide range of storage solutions designed to support different data types, access patterns, and analytical requirements. The purpose of this section is to provide an overview of Azure storage services and to explain the specific storage options selected for the Energy Solution project.

Azure Storage Account

An Azure Storage Account offers a comprehensive suite of storage services, each serving a different purpose within the solution architecture.

The following storage services are relevant to this project:

1. **Azure SQL DB**
   Azure SQL Database is a fully managed relational database service used to store structured and analytics-ready data. In this project, it serves as the Gold layer data warehouse for storing fact and dimension tables consumed by Power BI for reporting and analysis.

2. **Azure Data Lake Storage Gen2**
   Azure Data Lake Storage Gen2 serves as the primary data lake for the Energy Solution project. It stores data in Bronze, Silver, and Gold layers, enabling scalable storage, high-performance analytics, and seamless integration with Azure Data Factory and Power BI.

3. **File Storage**
   Azure File Storage provides a fully managed cloud file share accessible via the SMB protocol. While not a core component of this solution, it can be used for configuration files and shared reference data if required.

4. **Table Storage**
   Azure Table Storage stores structured, NoSQL key-value data. In this project, similar concepts are implemented through configuration and lookup tables used for rule-based scoring and metadata management.

2.3 Overview of Azure Data Factory

Azure Data Factory (ADF) is a cloud-based data integration service that orchestrates the movement and transformation of data between various data stores and compute resources. The primary purpose of Azure Data Factory is to extract data from one or more source systems, process and transform it as required, and load it into target systems for analysis and reporting. In the Energy Solution project, Azure Data Factory is used to automate data ingestion, perform transformations, and manage end-to-end data workflows efficiently.

Azure Data Factory consists of several key components, including Linked Services, which define connections to data sources and destinations; Datasets, which represent the structure of data within those sources; Pipelines, which act as logical containers for activities; Activities, which perform tasks such as data movement and transformation; and Integration Runtime, which provides the infrastructure required to execute data movement and processing. Together, these components enable scalable, reliable, and automated data processing within the project architecture.

3: ENVIRONMENTAL SET-UP:

3.1 Creating Azure Data Factory Resource via Azure Portal

Azure Data Factory is a cloud-based data integration service that allows users to create, schedule, and manage data workflows. The process of creating the ADF resource involves selecting a subscription, defining a resource group, choosing a region, providing the resource name, and configuring additional settings.

Step-by-Step Guide

Step 1. By Accessing the Azure Portal, open the Azure portal and log in to your Azure account.

Step 2. Creating a New Resource and click on "Create a Resource" from the top-left menu.

Step 3. In the Azure Marketplace, locate "Data Factory" under the "Integration" category.

Alternatively, use the search bar and enter "Data Factory" to find the service quickly.

Step 4. For Initiating Data Factory Creation, Click on "Create" to begin the process.

Choose the appropriate subscription for the Data Factory resource.
Create a new resource group for the project, to organize all resources.
Select the region closest to your location; for example, "UK South."

Step 5. Networking Configuration (Optional):

Accept the default networking settings unless specific configurations are required.

Step 6. Add relevant tags to the Data Factory if necessary.

Step 7. Review the information provided in the form to ensure accuracy.

Click "Create" to start the deployment process.

Step 8. The Azure Data Factory tool will open, providing options for "Author" and "Monitor." "Author" is where development work will take place, and "Monitor" allows for monitoring the Data Factory activities.

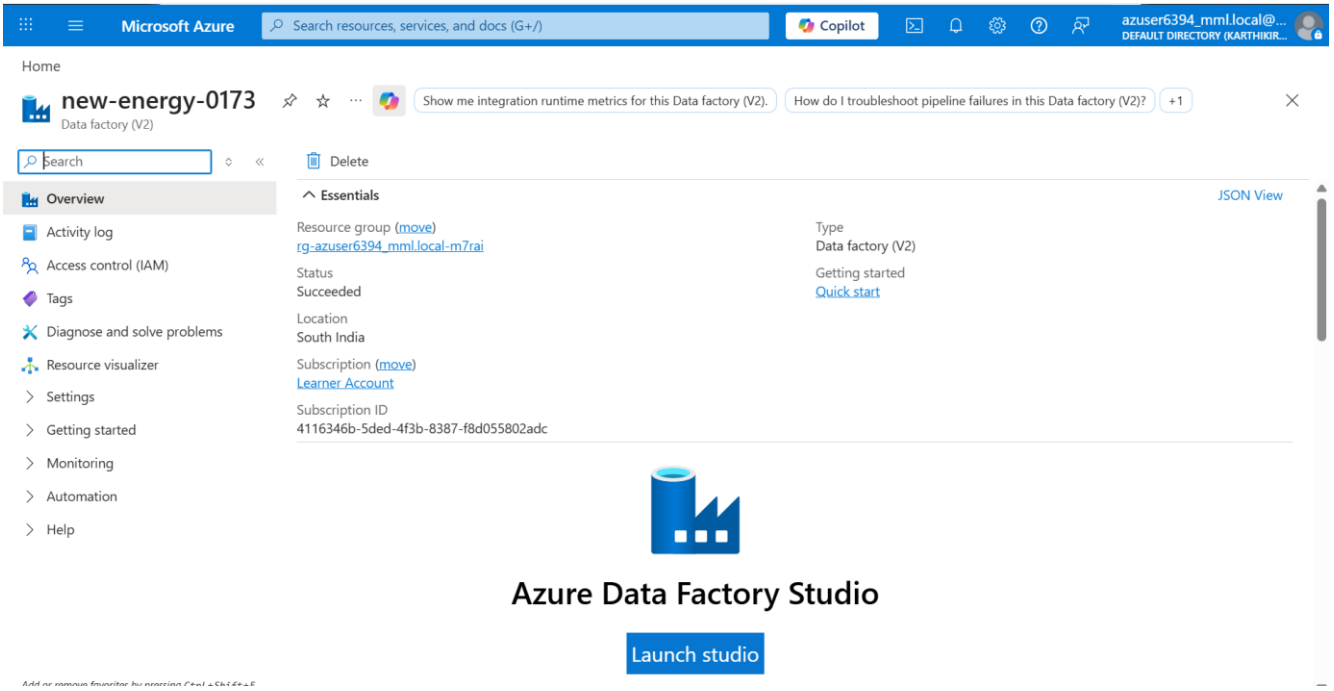


Fig.3.1: Data Factory Overview

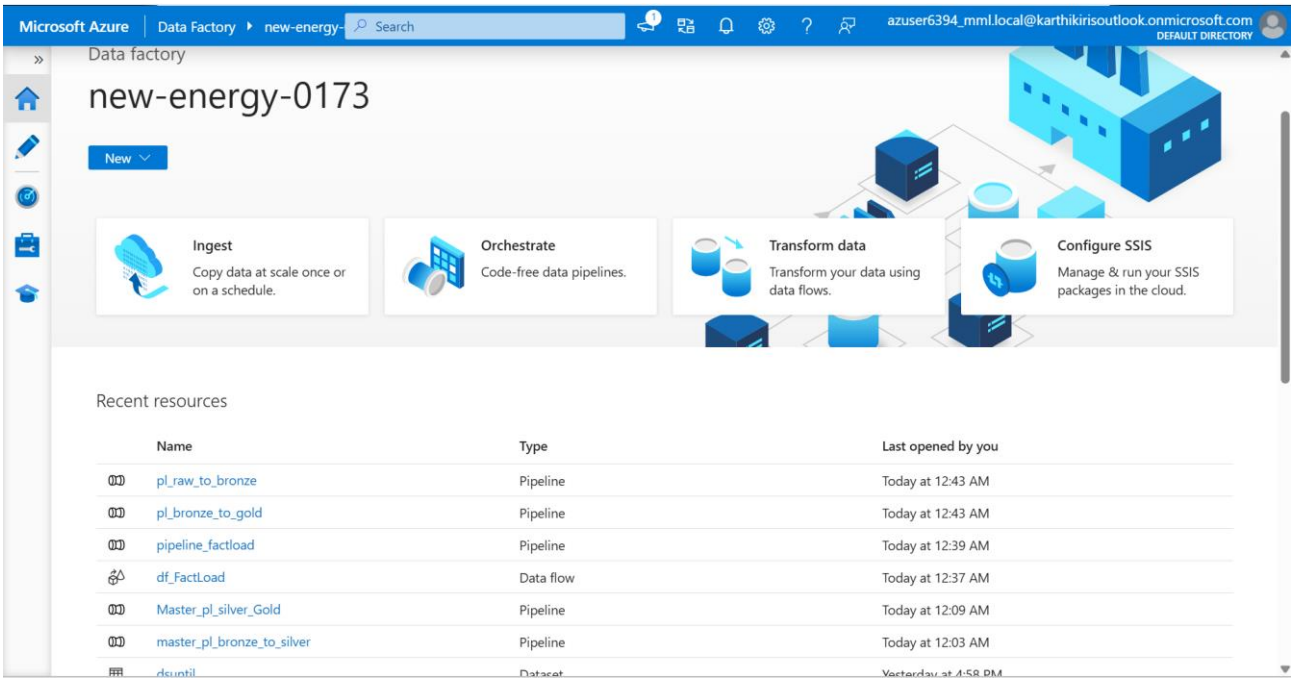


Fig.3.2: Azure Data Factory Studio

This report demonstrated the step-by-step process of creating an Azure Data Factory resource through the Azure portal. By following the outlined instructions, users can easily set up a Data Factory and access its development and monitoring functionalities.

3.2 Creating Azure Storage Account via Azure Portal

Azure Storage Account is a cloud-based storage solution that provides various options for storing and managing data, including containers, file shares, queues, and tables. The report guides readers through the necessary information required to set up the storage account, such as subscription selection, resource group allocation, naming conventions, replication options, and more.

Step-by-Step Guide

Step 1. Open the Azure portal and log in to your Azure account.

Step 2. Click on "Create a Resource" from the top-left menu.

Step 3. Locating Azure Storage Account:

In the Azure Marketplace, search for "Storage Account" and select the option from the results.

Step 4. Storage Account Information:

3.1.1 Begin the storage account creation process by providing necessary details:

3.1.2 Choose the appropriate subscription.

3.1.3 Select the previously created resource group.

3.1.4 Choose the closest region (e.g., UK South) for better performance. Select the "Locally Redundant Storage (LRS)" option for basic replication and cost-effectiveness.

Step 5. If not necessary, accept the defaults for advanced networking, data protection, encryption, and tags.

Step 6. Review the information provided to ensure its accuracy.Click "Create" to initiate the storage account deployment.

Step 7. Managing Access Control (IAM):

a. Utilize Access Control (IAM) to manage role-based access to the storage account.

b. Assign specific roles to team members to control their access.

Step 8. Accessing Keys:

a. Access the "Access keys" section to view and manage two sets of keys (key1 and key2) automatically generated for the storage account.

b. These keys are crucial for application authentication and data interaction.

Step 9. Interacting with Data:

a. Navigate to "Data Storage" in the menu to interact with data within the storage account.

b. Use the "Containers" section to manage containers and create new ones.

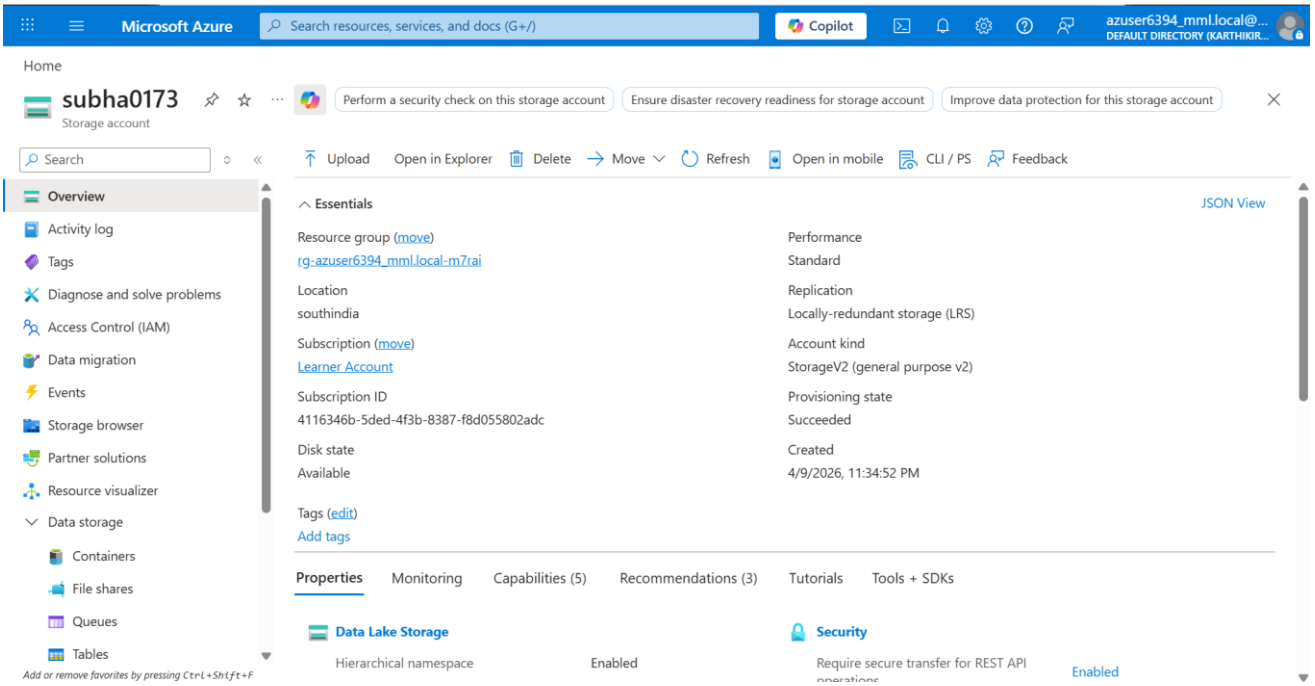


Fig.3.3: Azure Storage Account Overview

By following the provided instructions, users can efficiently set up a storage account and manage access control, access keys, and data interaction options. The Azure Storage Account provides a robust and flexible solution for storing and managing various types of data, making it an essential resource for cloud-based projects and applications.

3.3 Creating Azure Data Lake Storage Gen2 Account via Azure Portal

Azure Data Lake Storage Gen2 builds on top of the standard storage account, providing additional features like hierarchical namespace, fine-grain security, and compatibility with big data workloads. It is specifically designed to serve as a Data Lake for big data analytics projects.

Step-by-Step Guide

Step 1. Creating a New Data Lake Storage Gen2 Account, Click on "Create a Resource" from the top-left menu.

Step 2. Locating Azure Storage Account:

3.1.5 In the Azure Marketplace, search for "Storage Account" and select the option from the results.

Step 3. Data Lake Storage Gen2 Information:

a. Proceed with the creation process by providing the required details:

b. Choose the appropriate subscription. Select the previously created resource group.

c. Assign a unique name for the Data Lake Storage Gen2 account.

Step 4. Enabling Hierarchical Namespace:

a. In the "Advanced" section, enable the "Hierarchical Namespace" option to transform the storage account into a Data Lake Storage Gen2 account.

Step 5. Deployment Completion:

a. Wait for the deployment process to complete; it may take a minute.

b. Once deployment is successful, click "Go to Resource" to access the Data Lake Storage Gen2 account.

Step 6. Viewing Data Lake Properties:

a. The properties of the Data Lake Storage Gen2 account are listed, with the hierarchical namespace enabled.

b. The Data Lake Storage section is visible, differentiating it from a standard storage account.

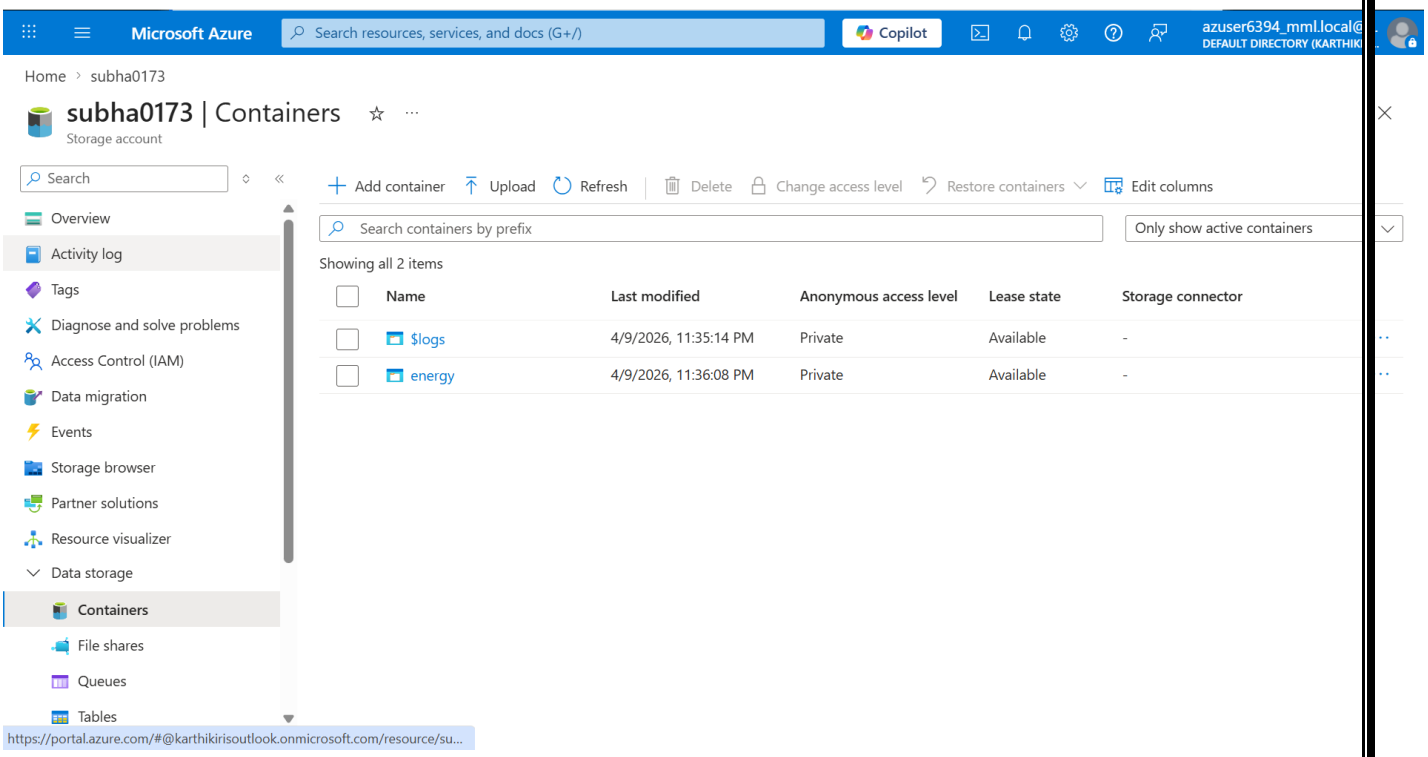


Fig.3.4: Data Lake Gen 2 Storage

By following the outlined steps, users can enable hierarchical namespace and access advanced features that make Data Lake Storage Gen2 suitable for big data analytics projects. Leveraging the hierarchical namespace, fine-grain security, and compatibility with big data workloads, Azure Data Lake Storage Gen2 proves to be a powerful solution for data engineers working on large-scale analytics projects.

3.4Creating Azure SQL Database via Azure Portal

Azure SQL Database is a fully managed cloud-based database that offers features such as SQL authentication, various service tiers, and configurable storage. The report provides a step-by-step guide on setting up the database and highlights specific functionalities relevant to SQL databases.

Step-by-Step Guide

Step 1. Creating a New Azure SQL Database:

3.4.1 Click on "Create a Resource" from the top-left menu.

Step 2. Locating Azure SQL Database:

a. In the Azure Marketplace, search for "SQL Database" and select the option from the results.

Step 3. Database Information:

a. Assign a unique name for the SQL Database.
b. Create a new database server with a unique name in the desired region (e.g., UK South).
c. Select SQL authentication for the authentication method.
d. Set a server admin username and password.

Step 4. Configure Database and Storage:

a. Select the workload environment as "Development" for cost-effectiveness.
b. Adjust the service tier to "Basic" based on a DTU-based purchasing model for a less demanding workload.

Step 5. Networking and Firewall Configuration:

a. Enable public endpoint connectivity for accessing the database from the internet.
b. Allow access to the database from other Azure services.
c. Add the current IP address to the firewall to enable local interaction with the database.

Step 6. Review the provided information for accuracy.

a. Click "Create" to initiate the creation of the Azure SQL Database.

Step 7. Query Editor (Optional):

a. Utilize the query editor to interact with the database, run SQL queries, and manage data.
b. Note that other desktop tools like Azure Data Studio and SQL Server Management Studio offer more features for extensive SQL work.

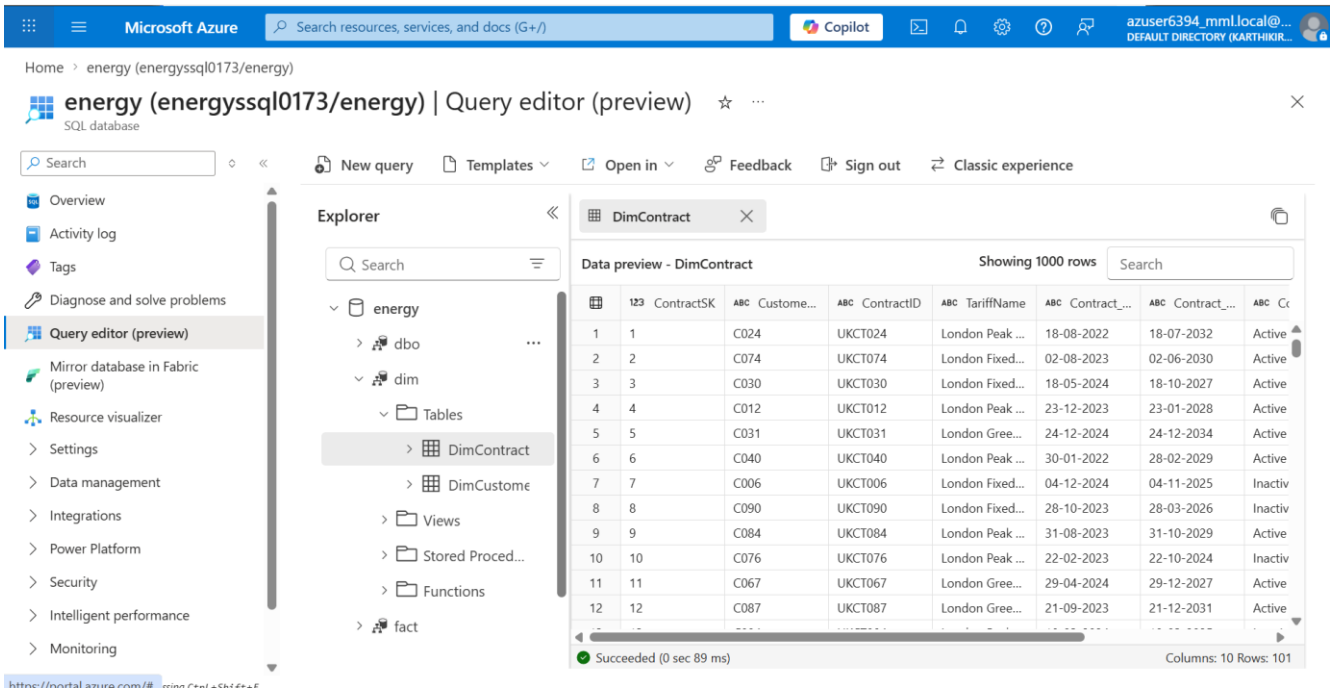


Fig.3.5: Azure SQL Database

Azure SQL Database offers fully managed cloud-based database services with multiple configuration options to suit various project needs. By following the outlined steps, users can create an efficient SQL database and leverage it for their development or production environments. Additionally, the report emphasized the availability of alternative desktop tools like Azure Data Studio and SQL Server Management Studio for enhanced SQL management and interaction.

3.5 Create Linked Service

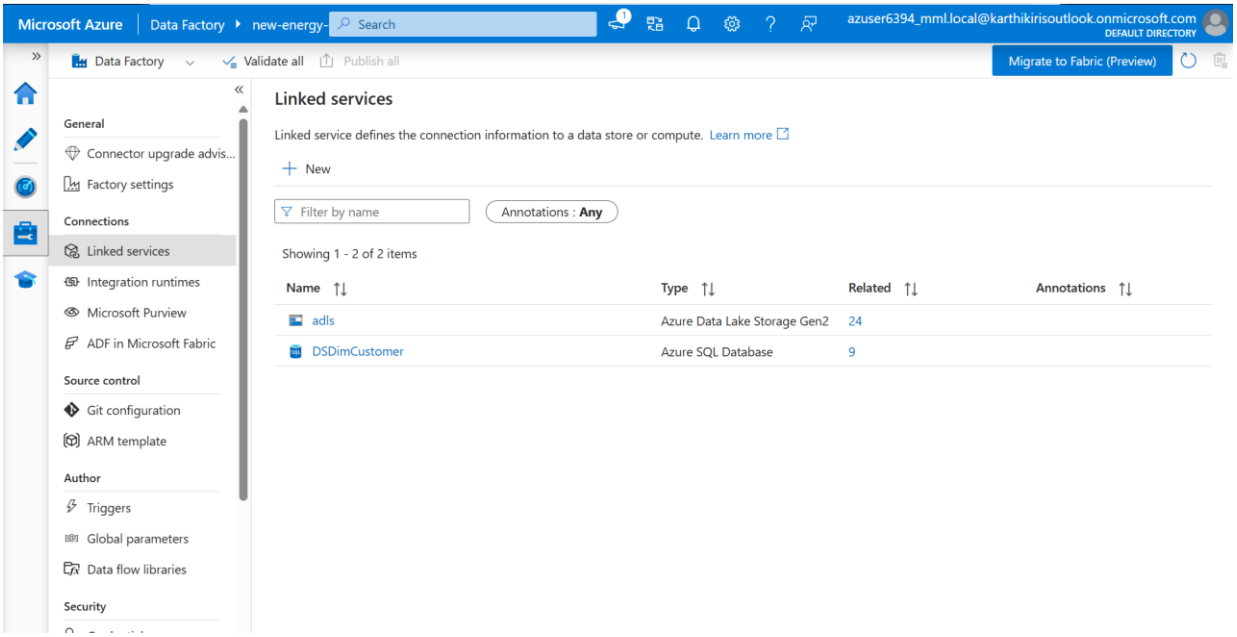
A Linked Service in Azure Data Factory defines the connection information required to connect to external data sources or destination systems. It acts as a bridge between Azure Data Factory and services such as Azure Data Lake Storage, Azure Blob Storage, or Azure SQL Database. Linked Services store authentication and connection details securely. They enable pipelines and datasets to access and move data across different systems.

Step-by-Step Guide

Step 1: Open Azure Data Factory

Log in to the Azure Portal and navigate to the required Azure Data Factory instance.

Step 2: Go to Manage Section



In the lefthand menu, click on Manage (toolbox icon).

Step 3: Select Linked Services

Under the Connections section, click on Linked services.

Step 4: Create New Linked Service

Click on + New to create a new linked service.

Step 5: Choose the Data Store Type

Select the required service type such as Azure Data Lake Storage Gen2, Azure Blob Storage, or Azure SQL Database, and click Continue.

Step 6: Configure Connection Details

Provide a meaningful name for the linked service, choose the Integration Runtime, select the Azure subscription, and choose the target storage or database account. Configure the authentication method (for example, Account Key or Managed Identity).

Step 7: Test the Connection

Click on Test connection to verify that the connection details are correct.

Step 8: Create the Linked Service

Once the test is successful, click Create to save the linked service.

Fig.3.6: Linked Service in ADF

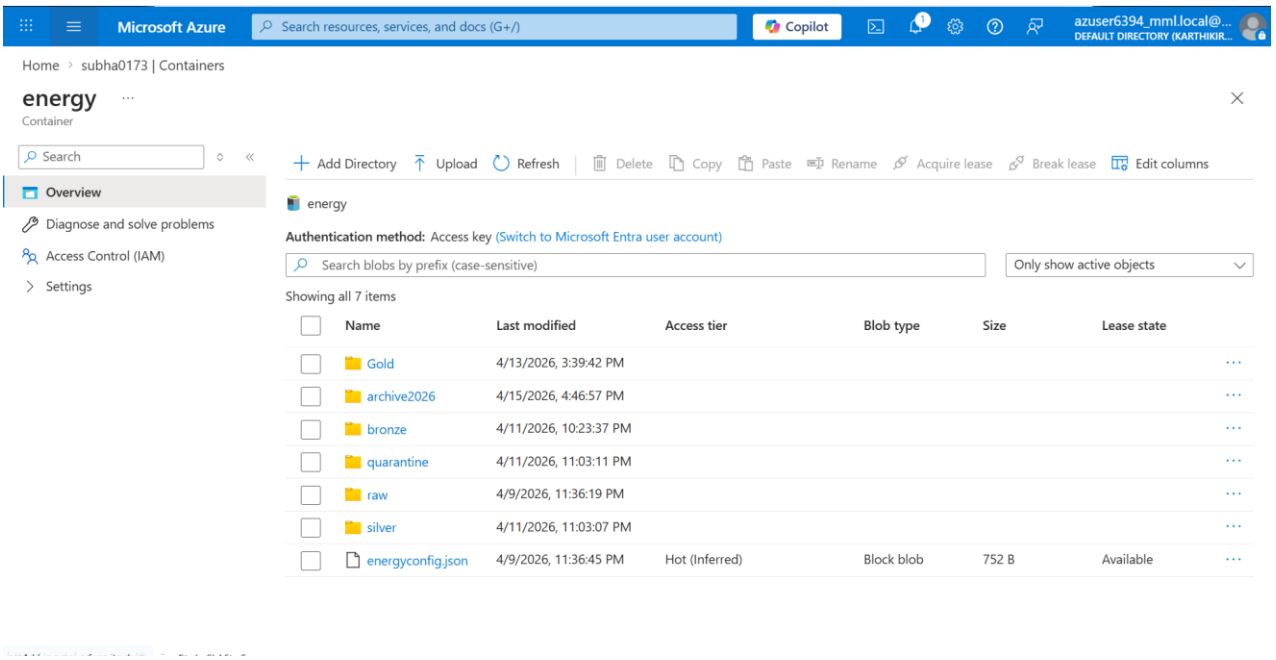


Fig.3.7 Data in Folder

4 IMPLEMENTATION

The implementation phase of the Energy Solution project focuses on converting the designed architecture into a fully functional data analytics system using Microsoft Azure services. This phase involves configuring storage services, developing Azure Data Factory pipelines, implementing data transformations, and loading analytics-ready data into the reporting layer.

The project is implemented using a layered approach, beginning with the ingestion of raw data from multiple source systems such as customer master, billing and payments, energy usage, customer interactions, and external market data. Azure Blob Storage is used as the source staging area, while Azure Data Lake Storage Gen2 serves as the central data lake for storing data in Bronze, Silver, and Gold layers.

Azure Data Factory is used as the primary orchestration and integration tool. Multiple pipelines are developed to automate data ingestion, validation, and transformation processes. Linked services are configured to securely connect Azure Data Factory with source and target systems. Datasets are defined to represent the structure and format of incoming and outgoing data.

During implementation, Mapping Data Flows are created to cleanse, standardize, and enrich the data based on business rules. Slowly Changing Dimension (SCD) Type 2 logic is applied to maintain historical records of customer and contract changes. Rule-based churn and loyalty scoring is then calculated and stored in fact tables.

Finally, the processed and analytics-ready data is loaded into Azure SQL Database, which acts as the reporting layer and serves as the data source for Power BI dashboards. The entire implementation is validated through testing and debugging to ensure data accuracy, reliability, and performance. This structured implementation ensures an automated, scalable, and efficient solution for customer churn and loyalty analytics

Source Transformation:

In this initial stage, the raw data is accessed and loaded into the system. The process follows these key steps:

Step 1. Understanding the Requirement:

The business and technical requirements for customer churn and loyalty analysis are clearly defined. This includes identifying required data elements from customer, billing, usage, and interaction datasets and understanding transformation objectives.

Step 2. Uploading Source Data Files:

Raw source data files such as customer master data, billing and payments data, energy usage

data, and customer interaction data are uploaded into Azure Blob Storage to serve as the initial data source.

Step 3. Creating Source Datasets:

Source datasets are created in Azure Data Factory to represent the structure and format of the uploaded files. These datasets define the connection to Azure Blob Storage using configured linked services.

Step 4. Data Preview and Validation:

The source data is previewed within Azure Data Factory to verify schema, data types, and record completeness. Validation checks are performed to ensure data accuracy and consistency before transformation.

Step 5. Future Steps:

After validation, the source data is passed to subsequent transformation stages, including cleansing, standardization, enrichment, and historization using Mapping Data Flows and SCD Type 2 logic, leading to analytics-ready datasets.

Derived Column Transformation:

The focus of this stage is on creating a new "ECDC year-week" column through the Derived Column transformation:

1. Identification of Business-Required Derived Columns
2. Definition of Calculation and Business Logic Rules
3. Creation of New Columns Using Expressions
4. Standardization and Data Type Conversion
5. Handling of Null and Default Values
6. Verification of Derived Column Accuracy
7. Preparation of Transformed Data for Downstream Processin

4.1 Bronze Layer Execution

The Bronze layer represents the initial stage of the data processing architecture and serves as the raw data ingestion layer. It contains source data exactly as received from various systems such as customer master, billing and payments, energy usage, customer interactions, and external market sources, without applying any transformations. Data is ingested into the Bronze layer using Azure Data Factory Copy activities and stored in Azure Data Lake Storage Gen2 while preserving original file formats and structures. This layer ensures data traceability, auditability, and replay capability, forming a reliable foundation for downstream Silver and Gold layer transformations.

Step to step procedure:

Step 1: Pipeline Triggering

The Bronze layer pipeline (pl_raw_to_bronze) is triggered either manually or through a scheduled trigger in Azure Data Factory. This pipeline is responsible for ingesting raw data from source storage.

Step 2: Lookup Activity Execution

A **Lookup activity** is executed first to read configuration or metadata information from the configuration dataset. This includes details such as source file paths, entity names, and target Bronze locations.

Step 3: Retrieval of Source File List

The output of the Lookup activity returns a list of source files or entities that need to be processed during the current pipeline run.

Step 4: ForEach Activity Initialization

The pipeline then enters a ForEach activity, which iterates over each source file or entity returned by the Lookup activity. This enables scalable and dynamic processing of multiple datasets.

Step 5: Copy Data Activity Execution

Inside the ForEach loop, a Copy Data activity is executed for each item. This activity copies data from the source system (such as Azure Blob Storage) into Azure Data Lake Storage Gen2 – Bronze layer.

Step 6: Raw Data Preservation

Data is copied in its original format without any transformation. Folder structures are maintained based on entity name and ingestion date to ensure traceability and auditability.

Step 7: Successful Completion and Logging

Once all files are processed, the pipeline completes successfully. Execution details such as run

status, duration, and data movement metrics are logged for monitoring and auditing purposes.

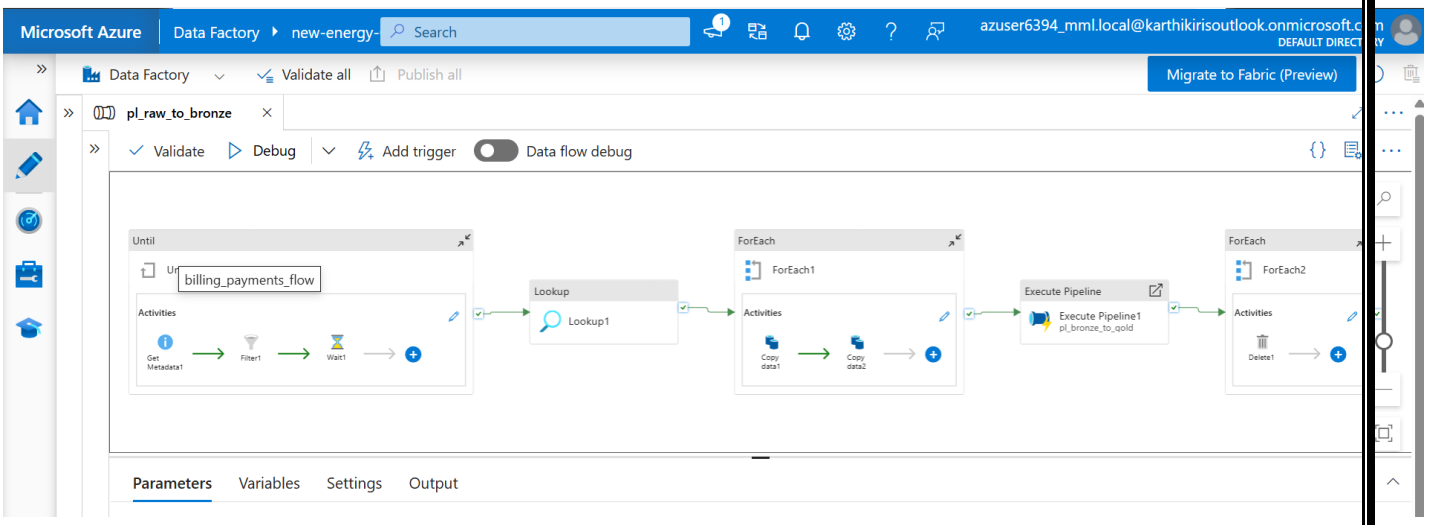


Fig.4.1.Bronze Layer Pipeline in ADF

4.2 Silver Layer Transformation

Silver Transformation involves cleansing, standardizing, and enriching raw data ingested into the Bronze layer. During this stage, data quality checks, validation rules, and business logic are applied using Azure Data Factory Mapping Data Flows. Derived columns and standardized formats are created to ensure consistency across datasets. The transformed data is stored in the Silver layer and prepared for SCD processing and analytics.

billing_payments_transformation

What it does:

This data flow transforms raw billing and payment data from the Bronze layer into clean, standardized Silver-layer data. It validates billing amounts, derives required metrics, and separates valid and invalid records. Clean records are written to the Silver layer, while invalid records are moved to quarantine.

Steps involved:

1. Add Source to read data from bronzebillingpayments.
2. Use Select to choose required columns (BillingID, CustomerID, BillAmount, etc.).
3. Apply Conditional Split to separate valid and invalid billing records.
4. Use Derived Column to create standardized billing fields.
5. Write valid data to silverbillingpayments and invalid data to quarantinebp.

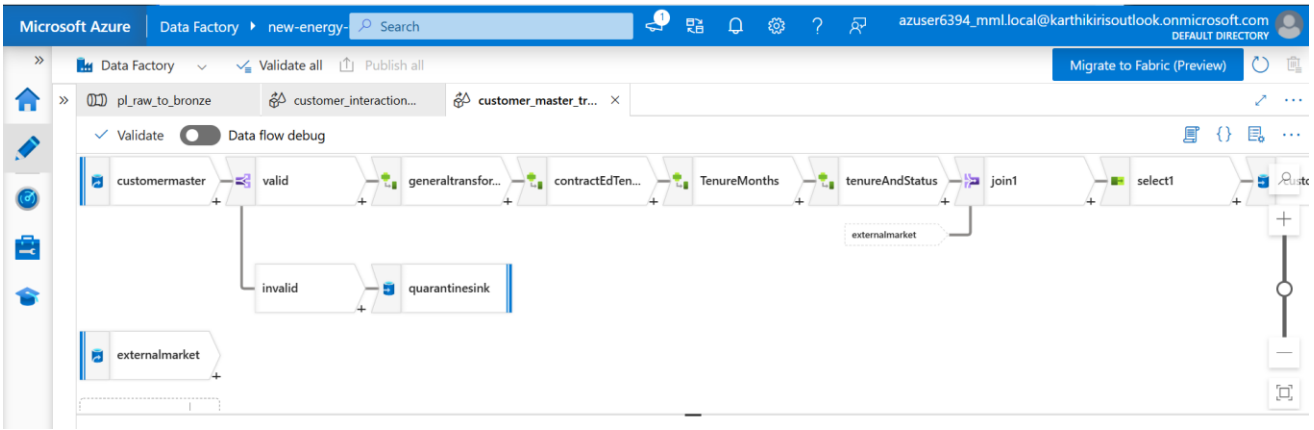


Fig.4.2.Silver Layer Data Transformation in ADF

customer_interactions_transformation

What it does:

This data flow processes raw customer interaction data such as complaints and service requests. It cleanses the data, categorizes interaction types, and prepares interaction metrics for churn analysis. Invalid interaction records are quarantined.

Steps involved:

1. Add Source to read data from bronzecustomerinteractions.
2. Apply Conditional Split to filter valid and invalid interaction records.
3. Use Derived Column to standardize and categorize interaction types.
4. Write valid records to silvercustomerinteractions.
5. Write invalid records to quarantineci.

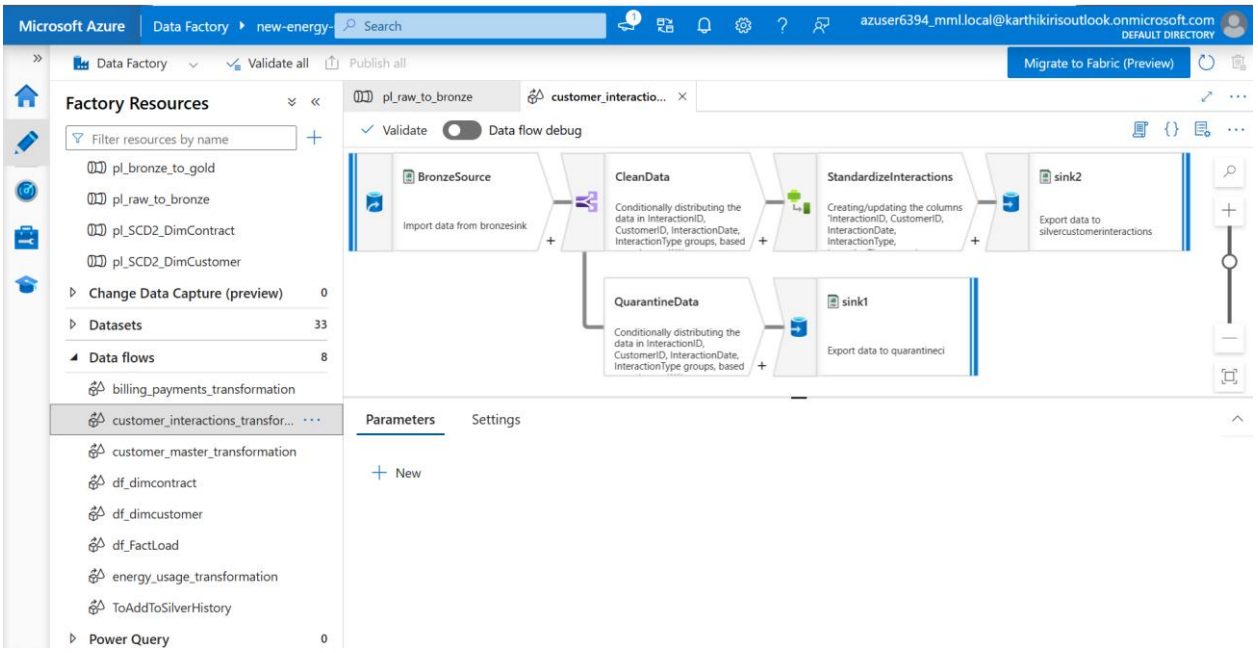


Fig.4.3.Customer Interactions Transformation in ADF

customer_master_transformation

What it does:

This data flow transforms customer master data by cleansing demographic fields, validating customer records, and deriving attributes such as tenure and status. It also enriches customer data using external market data before storing it in the Silver layer.

Steps involved:

1. Add Source to read data from bronzecustomermaster.
2. Apply Validation / Conditional Split to separate valid and invalid customers.
3. Use Derived Column to calculate tenure and normalize attributes.
4. Join with external market data for enrichment.
5. Write valid data to silversinkcm and invalid data to quarantinecm.

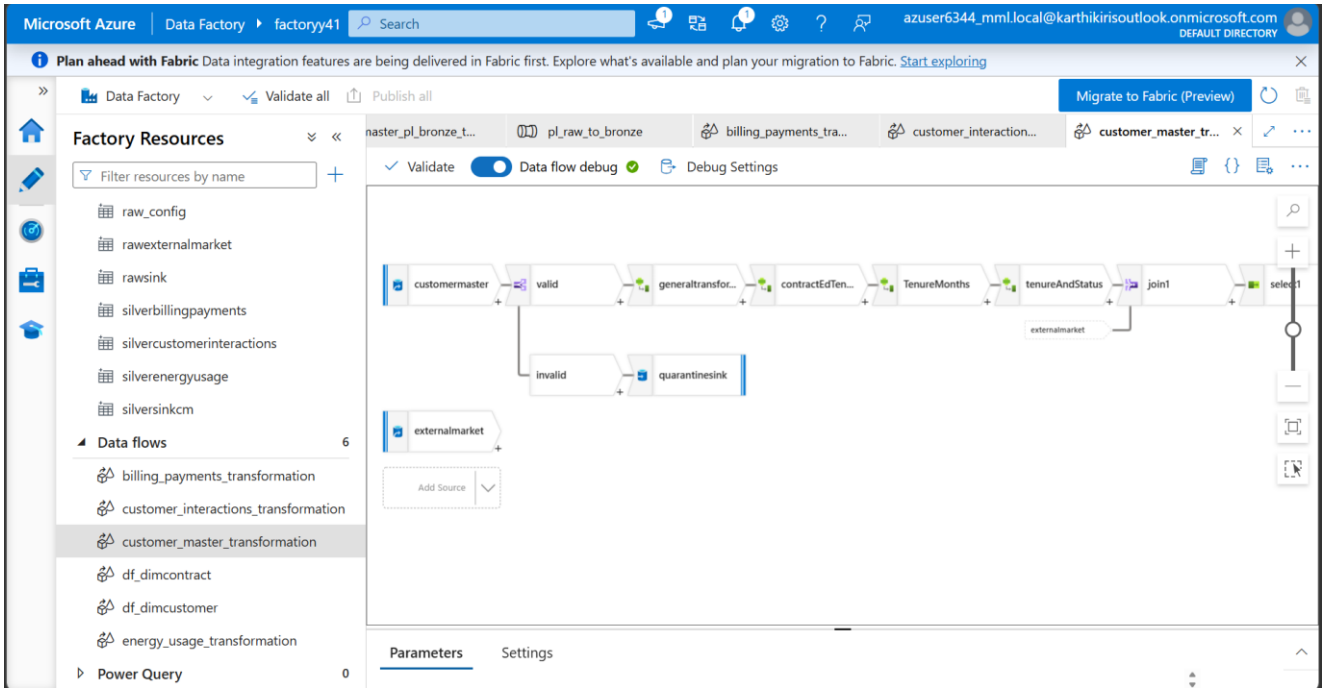


Fig.4.4.Customer Master Transformation in ADF

energy_usage_transformation

What it does:

This data flow processes raw energy consumption data to identify usage patterns and abnormalities. It standardizes usage values, derives consumption metrics, and prepares the data for analytical reporting and churn scoring.

Steps involved:

1. Add Source to read data from bronzeenergyusage.
2. Apply Conditional Split to filter valid and invalid usage records.
3. Use Derived Column to calculate standardized usage metrics.
4. Write valid records to silverenergyusage.
5. Write invalid records to quarantineeu.

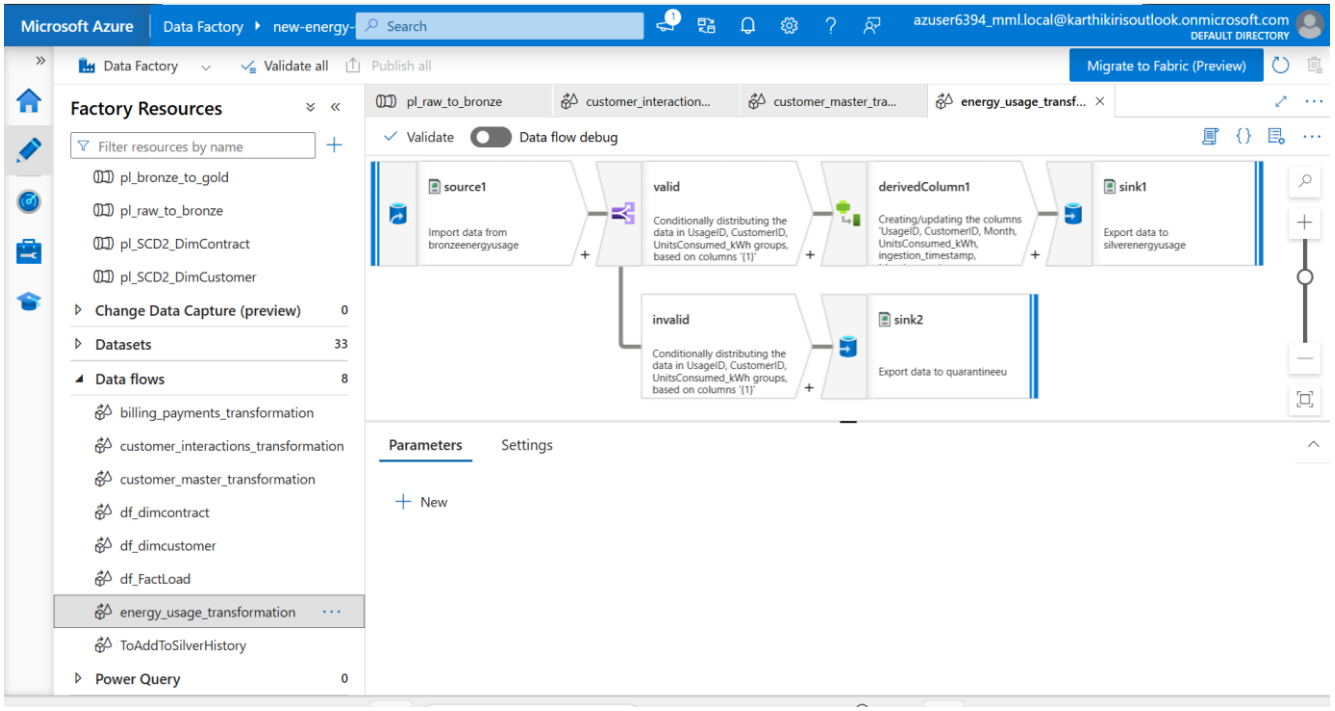


Fig.4.5.Energy Usage Transformation in ADF

4.3 SCD Type 2 Implementation

Slowly Changing Dimension (SCD) Type 2 is implemented in this project to maintain the full historical record of changes in key dimension tables such as Customer and Contract. Instead of overwriting existing records, SCD Type 2 creates a new version of the record whenever a change is detected, enabling point-in-time analysis.

In the Azure Data Factory Mapping Data Flow, the Silver-layer transformed data is first joined with the existing active dimension records from Azure SQL Database using business keys such as Customer ID or Contract ID. This join operation helps identify whether an incoming record is new, changed, or

unchanged when compared with the current active record.

Steps for Sdf_dimcustomer

Step 1: Source Data from Silver Layer

The transformed customer master data is read from the Silver layer dataset (silversinkcm) as the incoming source.

Step 2: Read Existing Dimension Records

The existing DimCustomer records are read from Azure SQL Database, filtering only active records (IsCurrent = 1).

Step 3: Join Incoming and Existing Records

A Join transformation is performed using the business key (CustomerID) to compare incoming customer records with existing dimension records.

Step 4: Identify Record Status

A Conditional Split is applied to classify records as New, Changed, or Unchanged based on attribute comparison.

Step 5: Apply Derived Column Logic

Derived columns are created to populate SCD fields such as EffectiveStartDate, EffectiveEndDate, and IsCurrent.

Step 6: Close Old Records

For changed records, the existing active record is updated by setting IsCurrent = 0 and updating the EffectiveEndDate.

Step 7: Insert New Records

New and changed customer records are inserted into the DimCustomer table with IsCurrent = 1.

Step 8: Load Data into Azure SQL DB

The final output is written to Azure SQL Database, maintaining complete historical customer records.

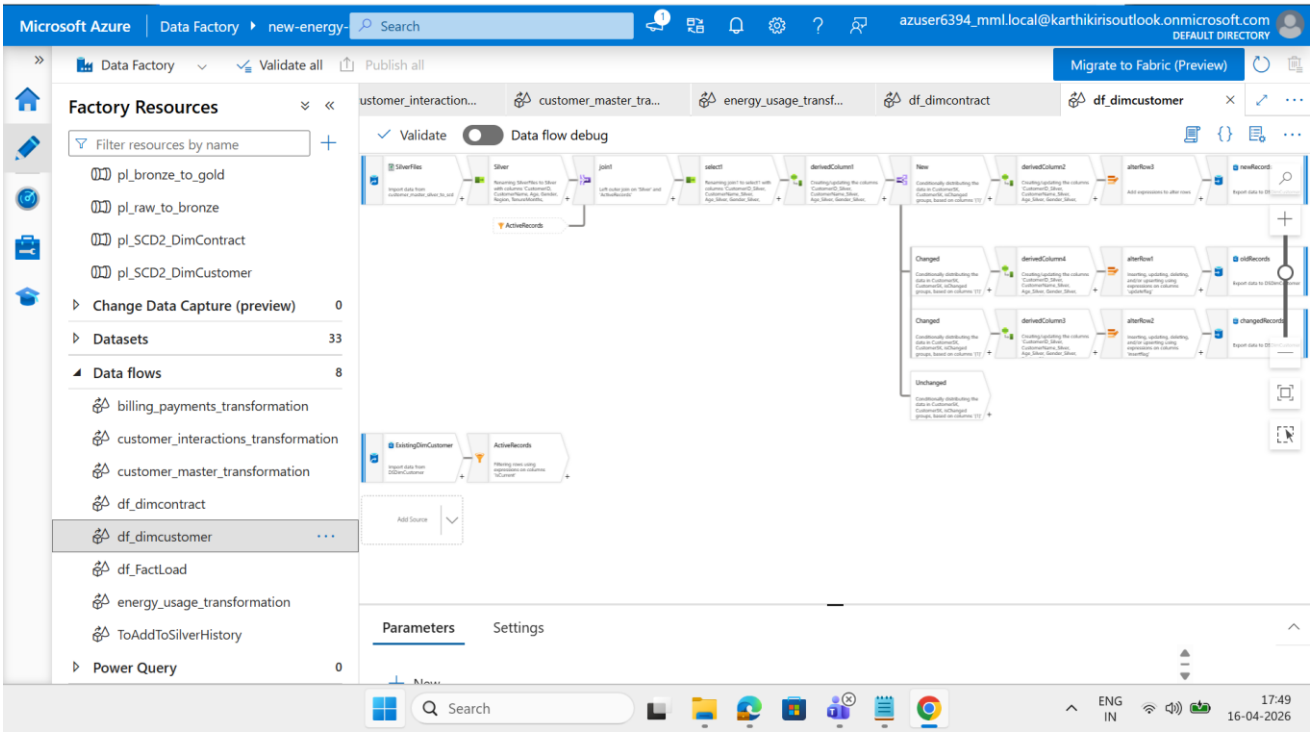


Fig.4.6.SCD Type 2 Implementation in df_dimcustomer

Steps for df_dimcontract

Step 1: Source Contract Data from Silver Layer

The cleansed contract and billing-related data is read from the Silver layer dataset.

Step 2: Read Existing Contract Dimension

Existing DimContract records are retrieved from Azure SQL Database, considering only active record

Step 3: Join Incoming and Existing Contract Data

A Join transformation is performed using ContractID to compare new contract data with existing dimension records.

Step 4: Detect Contract Changes

A Conditional Split identifies New, Changed, and Unchanged contract records based on attributes such as tariff, pricing plan, and contract status.

Step 5: Derive SCD Columns

Derived columns are created to populate EffectiveStartDate, EffectiveEndDate, and IsCurrent values for SCD handling.

Step 6: Expire Existing Records

For changed contracts, the current active record is expired by updating the end date and setting IsCurrent = 0.

Step 7: Insert Updated Contract Records

New versions of contract records are inserted with updated attributes and IsCurrent = 1.

Step 8: Persist Data to Azure SQL Database

The final dimension data is stored in Azure SQL Database, ensuring full contract history is maintain

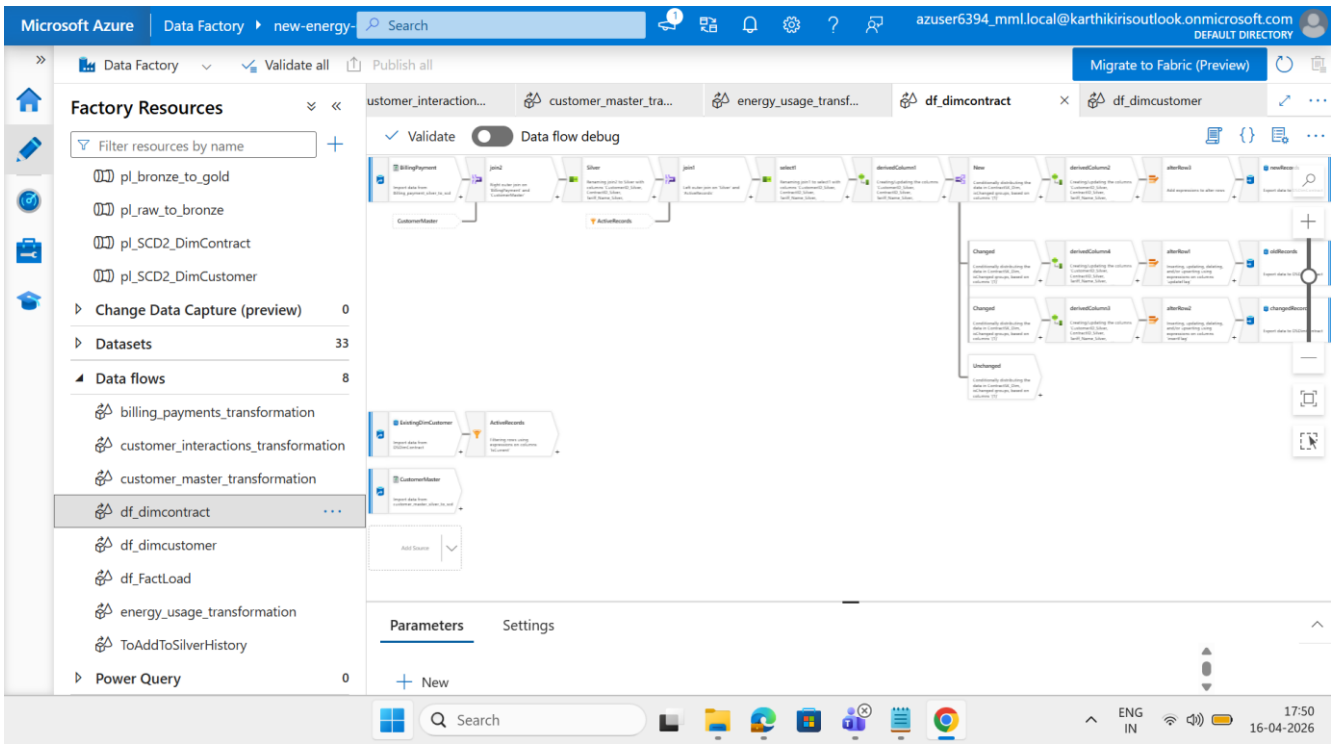


Fig.4.7.SCD Type 2 Implementation in df_dimcontract

4.4 Gold pipeline Transformation

df_dimcustomer

Step by step procedure:

Step 1: Read Silver Customer Data

The data flow starts by reading cleansed customer master data from the Silver layer dataset (silversinkcm).

Step 2: Read Existing DimCustomer Records

Existing records from the DimCustomer table are read from Azure SQL Database, filtering only active records where IsCurrent = 1.

Step 3: Join Incoming and Existing Records

A Join transformation is performed using the business key (CustomerID) to compare incoming Silver data with existing dimension records.

Step 4: Detect Record Changes

A Conditional Split classifies records into New, Changed, and Unchanged based on attribute-level comparison.

Step 5: Derive SCD Columns

Derived Column transformations populate SCD fields such as EffectiveStartDate, EffectiveEndDate, and IsCurrent.

Step 6: Expire Old Records

For changed records, existing active records are updated by setting IsCurrent = 0 and assigning the EffectiveEndDate.

Step 7: Insert New Customer Versions

New and changed customer records are inserted with IsCurrent = 1 and a new EffectiveStartDate.

Step 8: Write to Gold Layer (Azure SQL DB)

The final dimension output is written back to the DimCustomer table, preserving complete customer history.

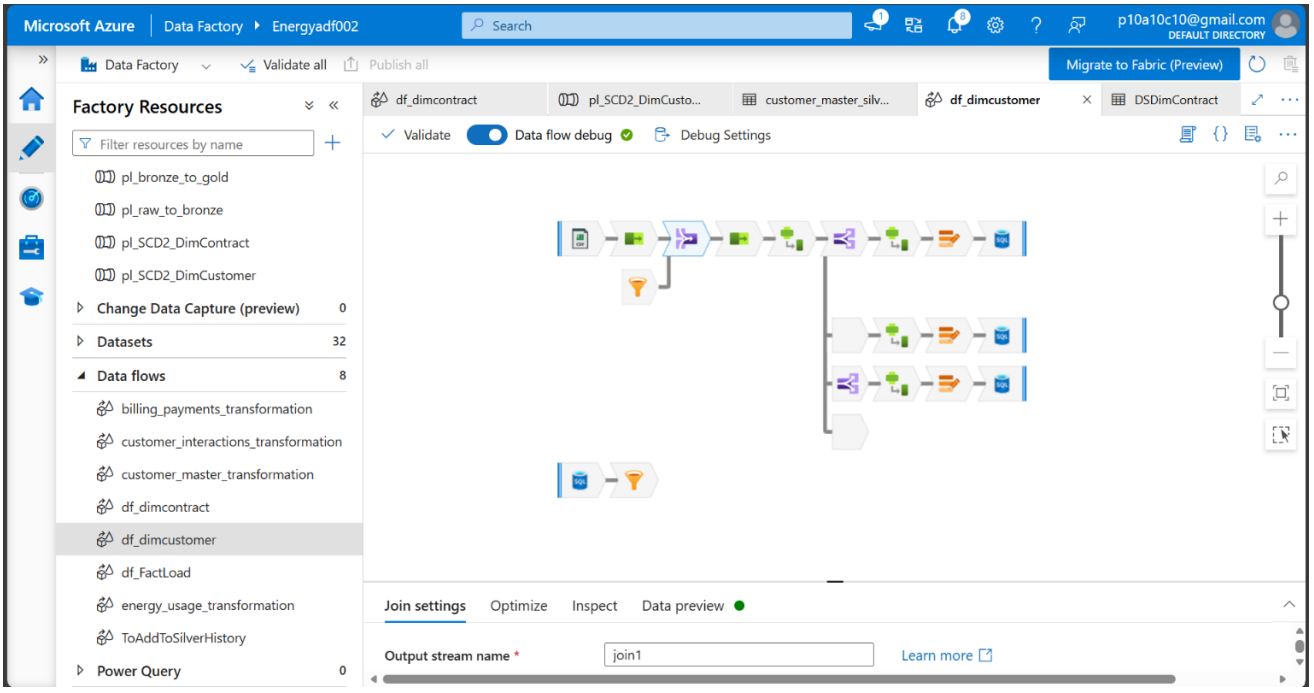


Fig.4.8 Dataflow Implementation in df_dimcustomer

df_dimcontract

Step 1: Read Silver Contract Data

The data flow begins by reading transformed contract and billing-related data from the Silver layer dataset.

Step 2: Read Existing DimContract Records

Existing active records from the DimContract table are read from Azure SQL Database.

Step 3: Join Contract Data

A Join transformation is performed using ContractID to compare incoming contract data with current dimension records.

Step 4: Identify Changes in Contract Attributes

A Conditional Split identifies New, Changed, and Unchanged contracts based on attributes such as tariff, pricing plan, and contract status.

Step 5: Apply Derived Column Logic

SCD columns (EffectiveStartDate, EffectiveEndDate, IsCurrent) are derived using business rules.

Step 6: Close Existing Contract Records

For changed contracts, current records are expired by setting IsCurrent = 0 and updating the end date.

Step 7: Insert Updated Contract Records

New versions of contract records are inserted with updated attributes and marked as active.

Step 8: Persist Data to Gold Layer

The final output is written to Azure SQL Database, ensuring full historical tracking of contract changes.

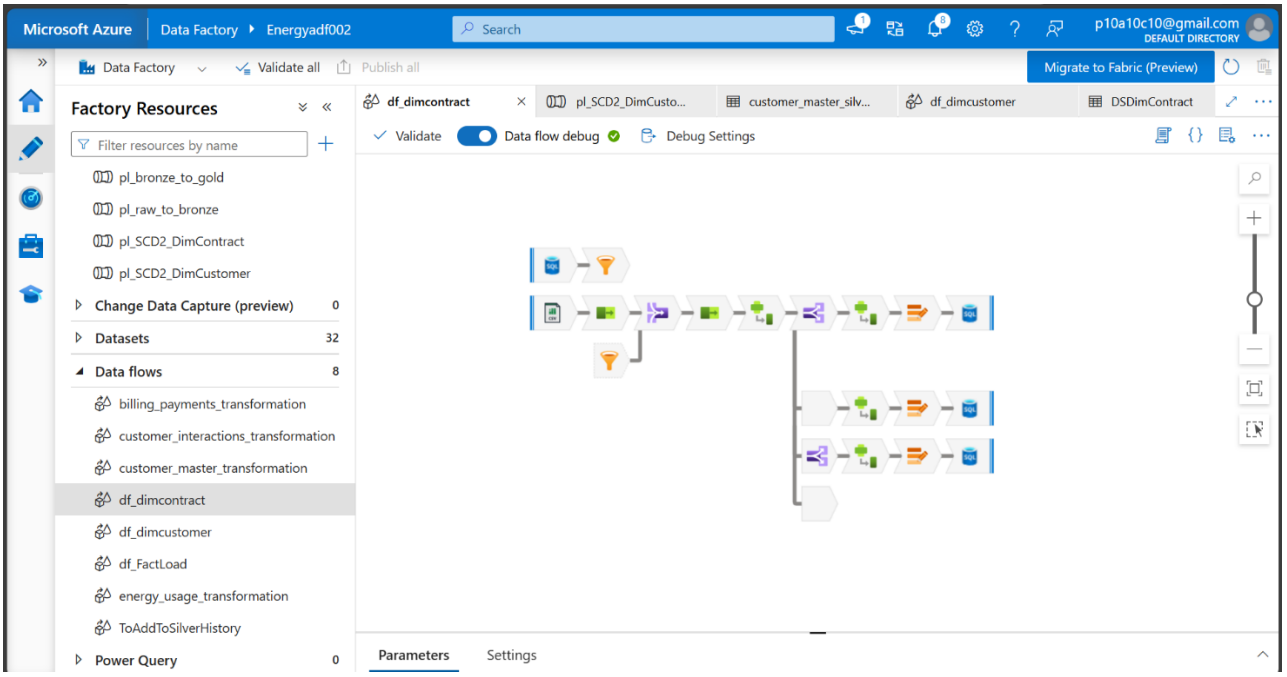


Fig.4.9 Dataflow Implementation in df_dimcontract

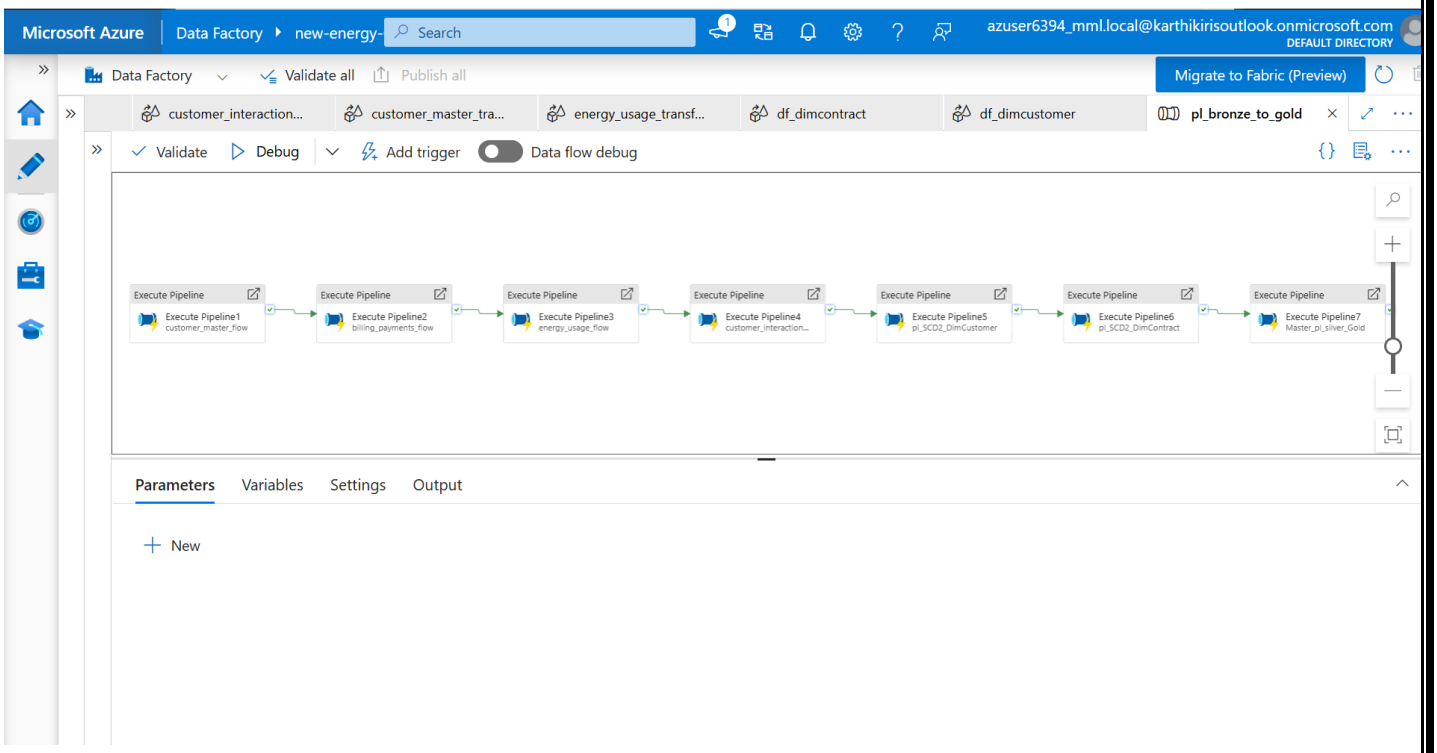


Fig.4.10 Pipeline pl_bronze_to_gold

Gold Pipeline step by step procedure:

Step 1: Pipeline Triggering

The Gold layer pipeline is triggered manually or through a scheduled trigger to initiate the final data processing stage.

Step 2: Until Activity Execution

An Until activity is used to control execution flow and ensure required Silver-layer data is available before proceeding further.

Step 3: Lookup Activity

A Lookup activity retrieves configuration or metadata details such as fact table names or processing parameters required for Gold layer loading.

Step 4: First ForEach Activity

The first ForEach activity iterates over the list of fact entities or configurations returned by the Lookup activity.

Step 5: Execute Pipeline Activity

Within the ForEach loop, an Execute Pipeline activity triggers the Gold layer data flow pipeline (pipeline_factload) to process each fact table.

Step 6: Second ForEach Activity

A second ForEach activity is used to handle additional post-processing steps or multiple fact table

executions independently.

Step 7: Successful Completion

After all iterations and executions are completed, the pipeline finishes successfully, and execution details such as run status and duration are logged.

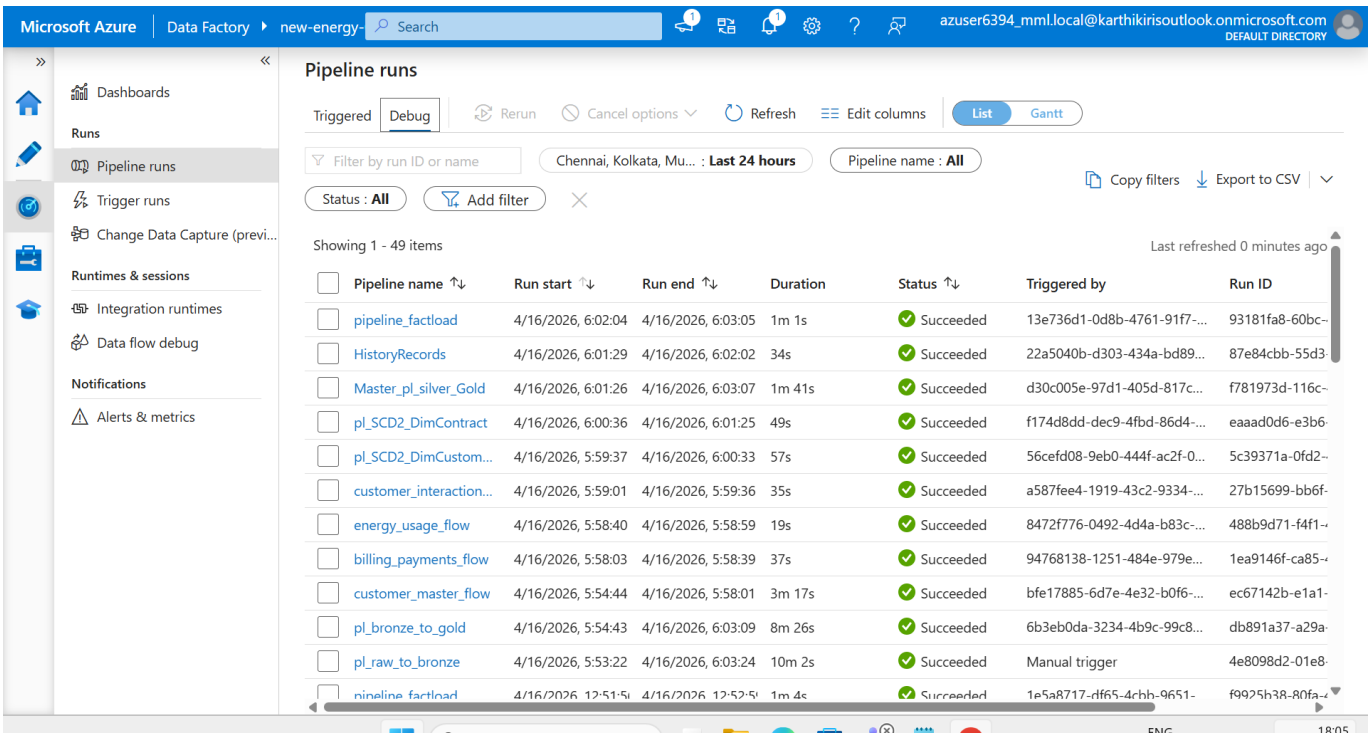


Fig.4.11 Gold Implementation Pipeline

Data Flow step by step procedure:

Step 1: Source Data from Silver Layer

The Gold layer data flow begins by reading cleansed and standardized data from the **Silver layer datasets**, including customer, billing, energy usage, and interaction data.

Step 2: Join with Dimension Tables

Silver-layer fact data is joined with **dimension tables** such as DimCustomer and DimContract using surrogate keys. This ensures referential integrity and prepares the data for analytical processing.

Step 3: Apply Business Logic and Calculations

Derived Column transformations are used to calculate business metrics such as churn risk indicators, loyalty scores, usage variance, billing trends, and interaction counts.

Step 4: Conditional Processing of Records

Conditional Split transformations are applied where required to segregate valid and invalid records or handle logic-based branching during fact computations.

Step 5: Aggregation and Enrichment

Aggregation transformations compute summarized metrics at the required grain (for example, customer-level or contract-level). Data is enriched using lookup and join transformations.

Step 6: Prepare Fact Table Structure

The transformed data is aligned to the target **fact table schema** by selecting required columns, reordering fields, and applying final data type conversions.

Step 7: Load Data into Gold Layer Sink

The final output is written to **Azure SQL Database** fact tables using sink transformations. Each fact table references the appropriate SCD-controlled dimension keys.

Step 8: Successful Execution and Validation

Once all transformations complete successfully, the Gold layer data flow finishes execution. Row counts and execution status are validated to ensure data completeness and correctness.

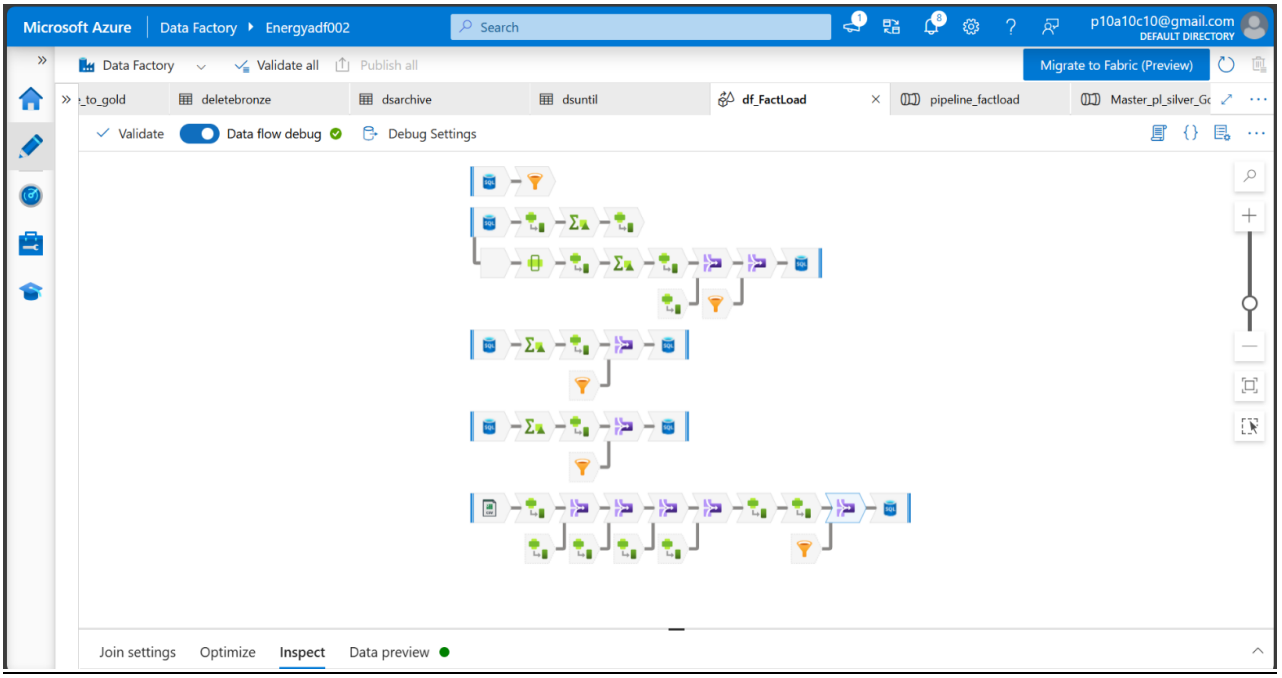
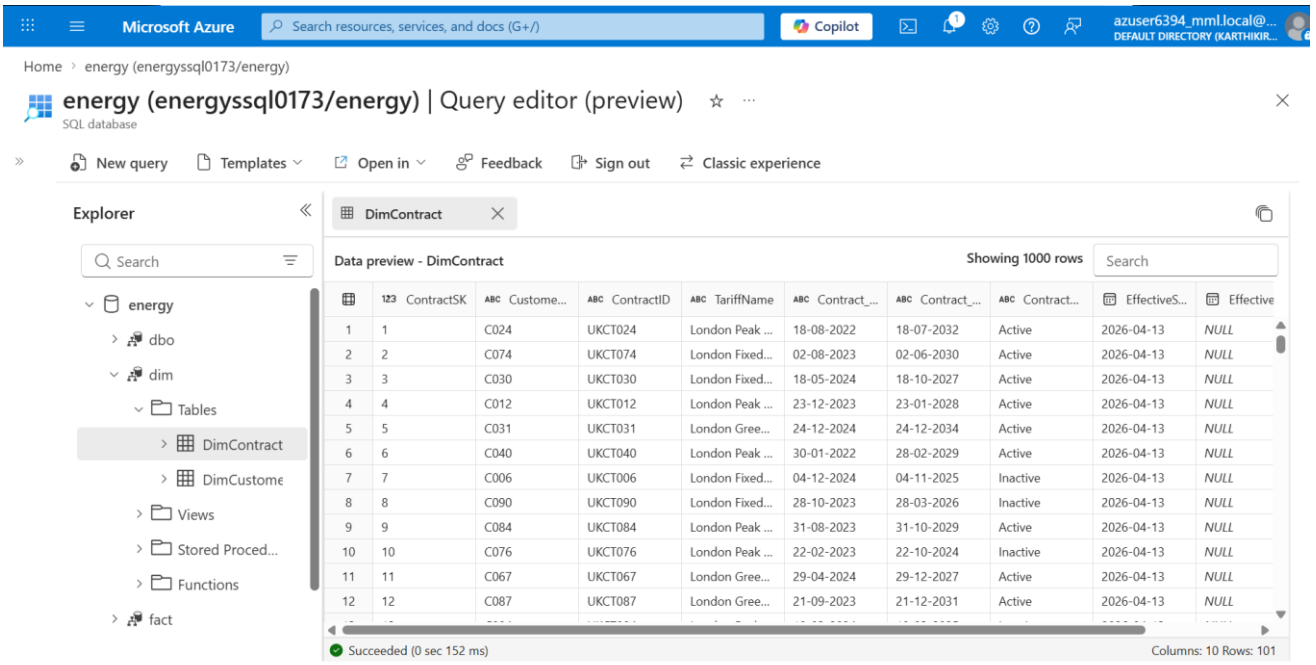


Fig.4.12 Gold Implementation Dataflow

5.RESULT:

After applying Slowly Changing Dimension (SCD) Type 2, the project generates structured dimension and fact tables that accurately preserve historical data changes. Dimension tables such as DimCustomer and DimContract maintain multiple versions of records whenever key attributes change, using fields like EffectiveStartDate, EffectiveEndDate, and IsCurrent to track record validity over time. The fact tables, including FactCustomerRisk, FactBillingBehaviour, FactUsageBehaviour, and FactCustomerInteractions, reference these dimensions through surrogate keys to ensure historical consistency. This design enables reliable timebased analysis, supports audit requirements, and ensures accurate interpretation of customer behaviour- and contractual changes.



	ContractSK	Custome...	ContractID	TariffName	Contract_...	Contract_...	Contract...	EffectiveS...	Effective
1	1	C024	UKCT024	London Peak ...	18-08-2022	18-07-2032	Active	2026-04-13	NULL
2	2	C074	UKCT074	London Fixed...	02-08-2023	02-06-2030	Active	2026-04-13	NULL
3	3	C030	UKCT030	London Fixed...	18-05-2024	18-10-2027	Active	2026-04-13	NULL
4	4	C012	UKCT012	London Peak ...	23-12-2023	23-01-2028	Active	2026-04-13	NULL
5	5	C031	UKCT031	London Gree...	24-12-2024	24-12-2034	Active	2026-04-13	NULL
6	6	C040	UKCT040	London Peak ...	30-01-2022	28-02-2029	Active	2026-04-13	NULL
7	7	C006	UKCT006	London Fixed...	04-12-2024	04-11-2025	Inactive	2026-04-13	NULL
8	8	C090	UKCT090	London Fixed...	28-10-2023	28-03-2026	Inactive	2026-04-13	NULL
9	9	C084	UKCT084	London Peak ...	31-08-2023	31-10-2029	Active	2026-04-13	NULL
10	10	C076	UKCT076	London Peak ...	22-02-2023	22-10-2024	Inactive	2026-04-13	NULL
11	11	C067	UKCT067	London Gree...	29-04-2024	29-12-2027	Active	2026-04-13	NULL
12	12	C087	UKCT087	London Gree...	21-09-2023	21-12-2031	Active	2026-04-13	NULL

Fig.5.1 Dimension table dim_contract

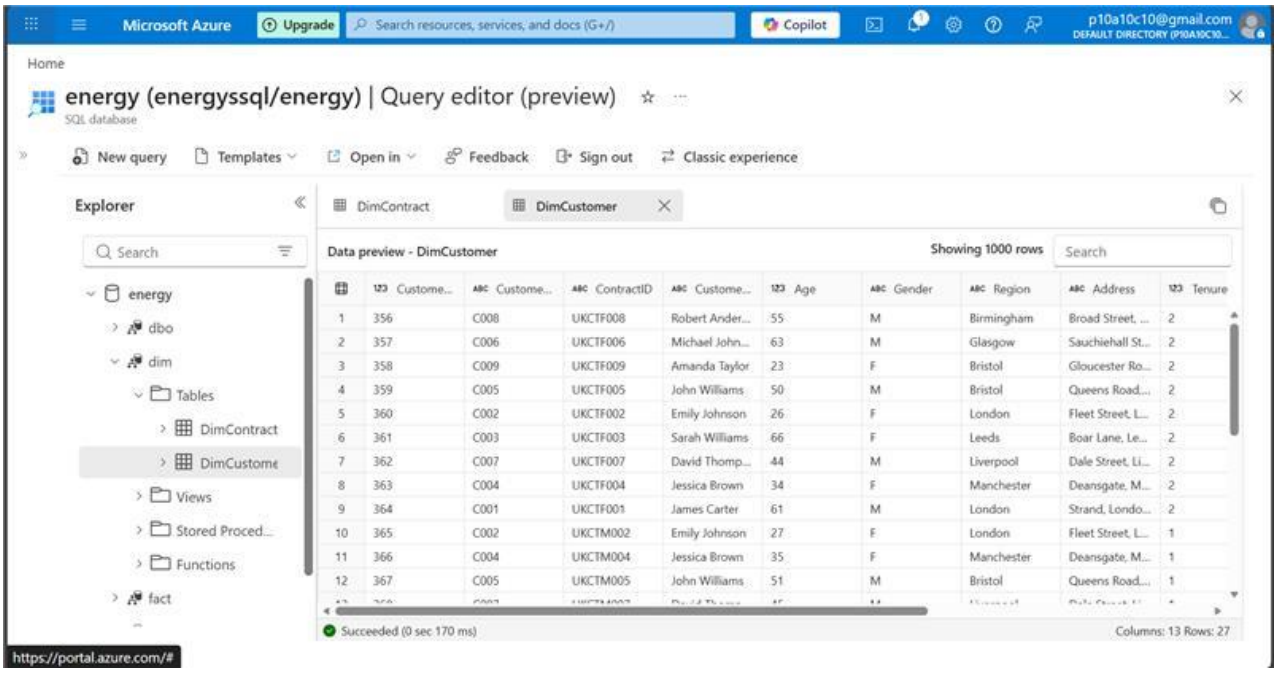


Fig.5.2 Dimension table dim_customer

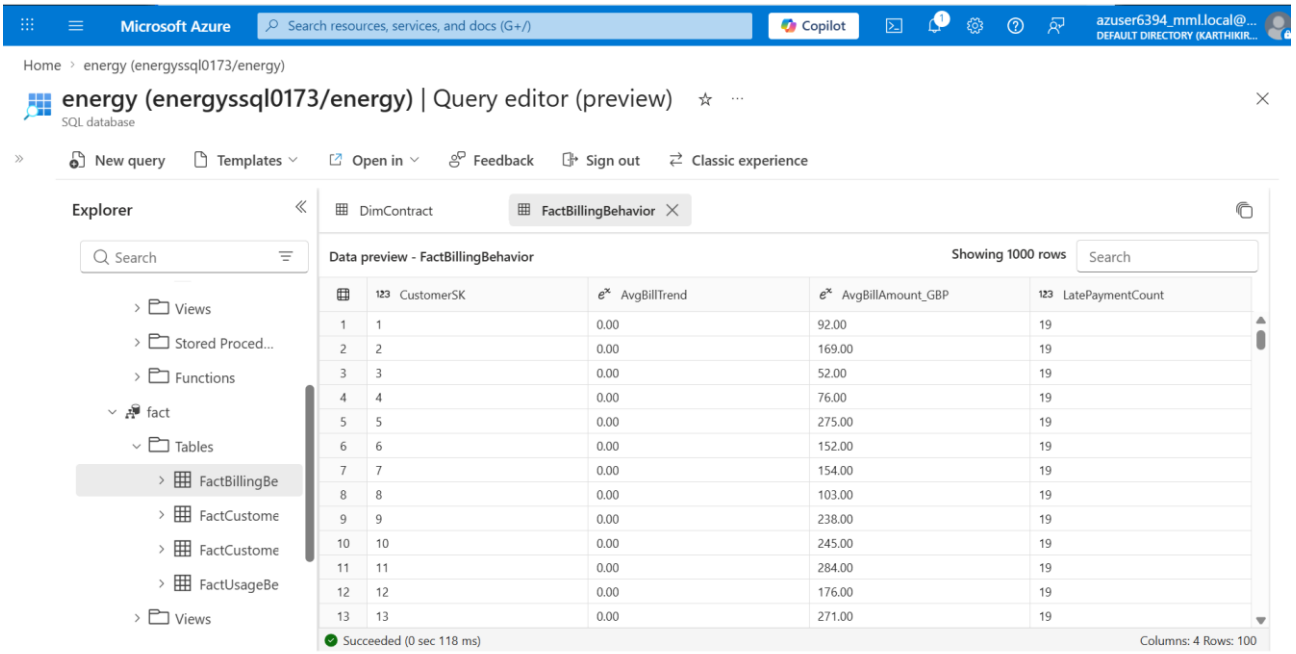


Fig.5.3 Fact table ds_FactBillingbehaviour

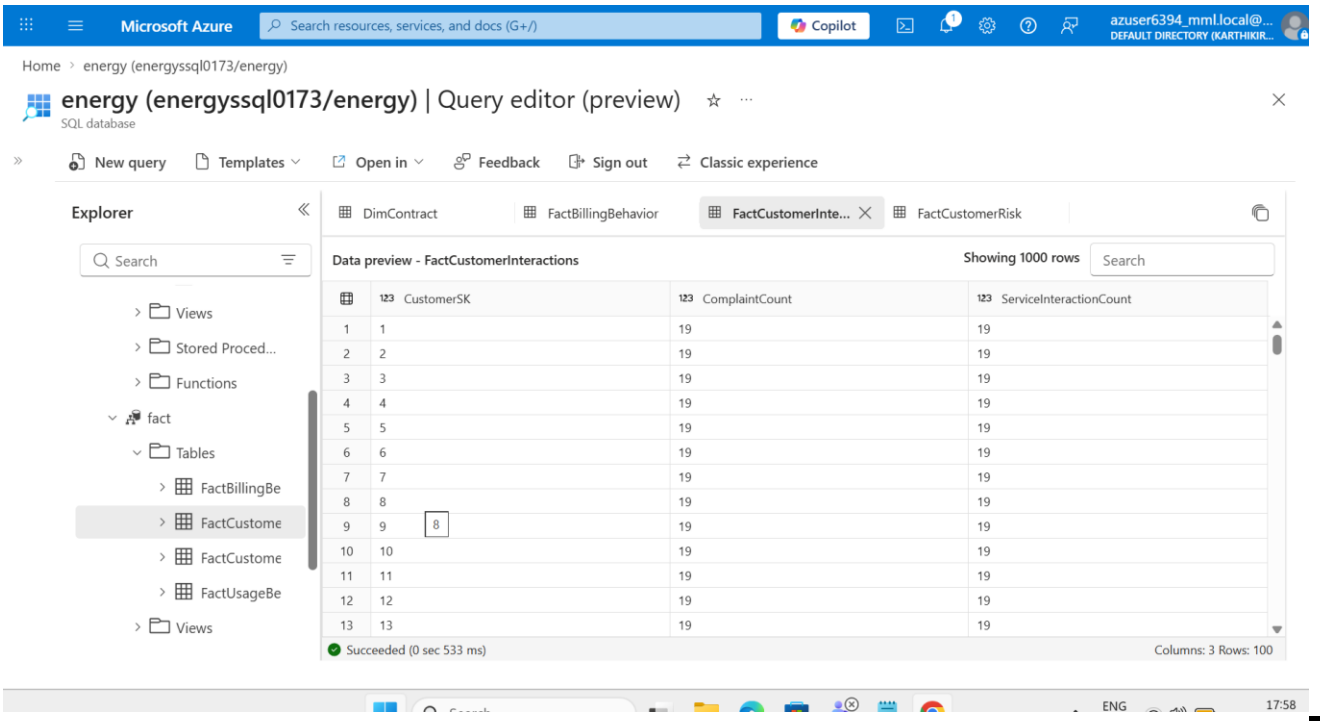


Fig.5.5 Fact table ds_CustomerInteractions

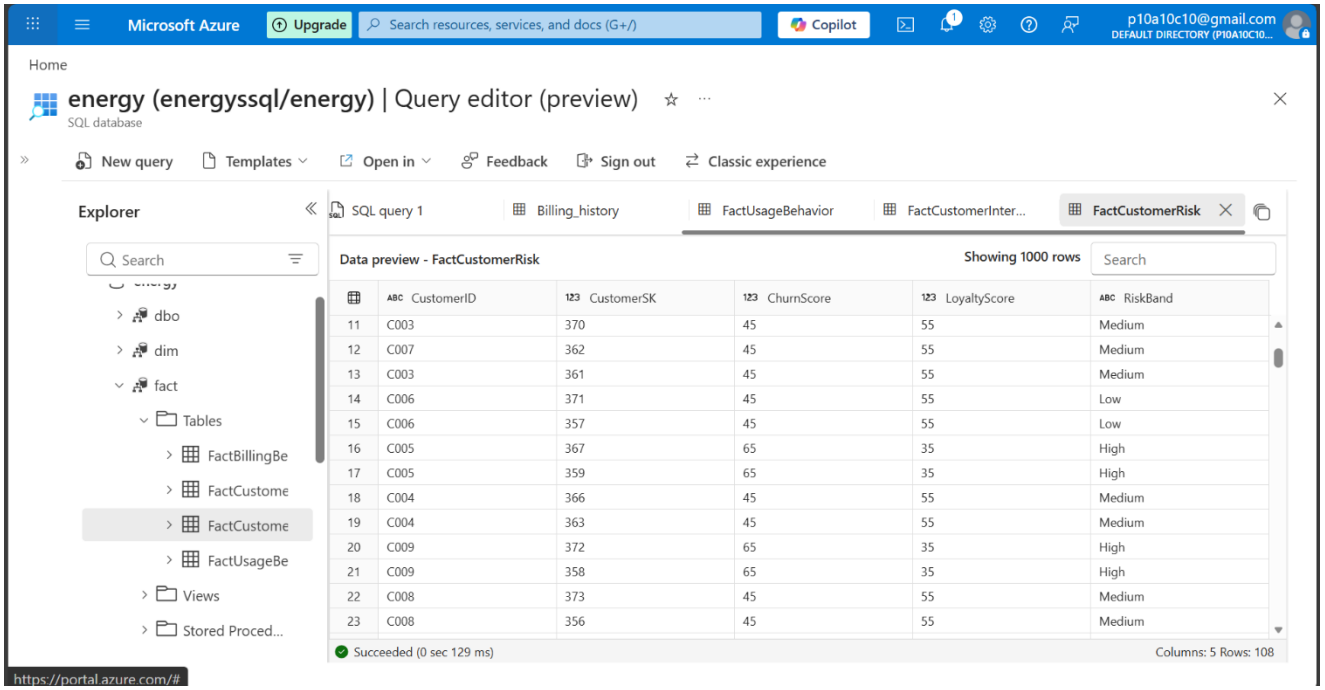
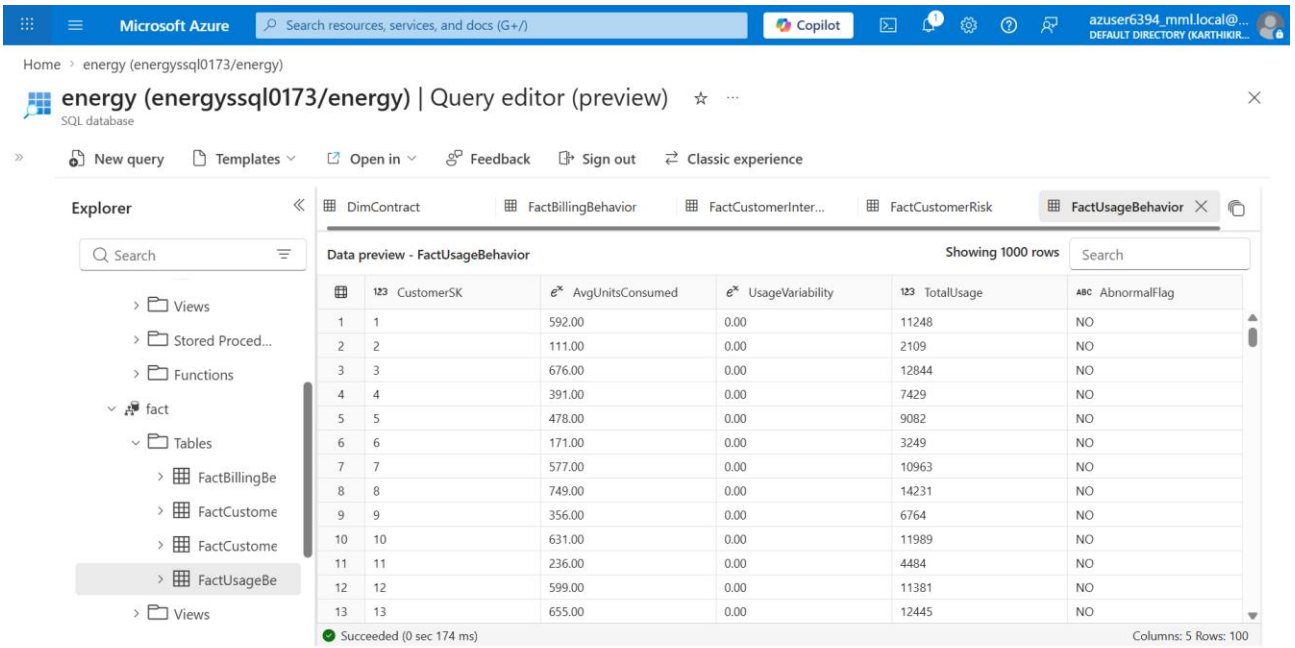


Fig.5.4 Fact table ds_Customerrisk



	CustomerSK	AvgUnitsConsumed	UsageVariability	TotalUsage	AbnormalFlag
1	1	592.00	0.00	11248	NO
2	2	111.00	0.00	2109	NO
3	3	676.00	0.00	12844	NO
4	4	391.00	0.00	7429	NO
5	5	478.00	0.00	9082	NO
6	6	171.00	0.00	3249	NO
7	7	577.00	0.00	10963	NO
8	8	749.00	0.00	14231	NO
9	9	356.00	0.00	6764	NO
10	10	631.00	0.00	11989	NO
11	11	236.00	0.00	4484	NO
12	12	599.00	0.00	11381	NO
13	13	655.00	0.00	12445	NO

Fig.5.6 Fact table ds_Usage Behaviour

6. CONCLUSION

The Energy Solution project successfully demonstrates the design and implementation of a scalable data analytics platform using modern Azure services. By adopting a layered architecture consisting of Bronze, Silver, and Gold layers, the project ensures efficient data ingestion, cleansing, transformation, and historical tracking. Azure Data Factory plays a central role in orchestrating data pipelines, enabling automation, reliability, and maintainability across the system.

The implementation of Slowly Changing Dimension (SCD) Type 2 ensures accurate historical tracking of customer and contract data, supporting time-based analysis and audit requirements. Rule-based transformations and structured fact and dimension tables provide a strong foundation for churn and loyalty analysis. Overall, the project meets its objectives by delivering a robust, automated, and analytics-ready solution that supports informed decision-making and demonstrates best practices in cloud-based data engineering.